

GNU Guix

GNU Guix

GNU Guix

Copyright © 2019, 2022 Ricardo Wurmus
Copyright © 2019 Efraim Flashner
Copyright © 2019 Pierre Neidhardt
Copyright © 2020 Oleg Pykhalov
Copyright © 2020 Matthew Brooks
Copyright © 2020 Marcin Karpezo
Copyright © 2020 Brice Waegeneire
Copyright © 2020 André Batista
Copyright © 2020 Christine Lemmer-Webber
Copyright © 2021 Joshua Branson
Copyright © 2022, 2023 Maxim Cournoyer
Copyright © 2023–2026 Ludovic Courtès
Copyright © 2023 Thomas Jeong
Copyright © 2024 Florian Pelz
Copyright © 2025 45mg
Copyright © 2023 Marek Felšöci
Copyright © 2023 Konrad Hinsén
Copyright © 2023 Philippe Swartvagher

, / GNU (GNU FDL) 1.3 , (FSF), , . "GNU Free Documentation
Licenses".

Table of Contents

1	Scheme	1
1.1	Scheme	1
2		5
2.1		5
2.1.1	“Hello World”	5
2.1.2		8
2.1.2.1		8
2.1.2.2		9
2.1.2.3		10
2.1.3		11
2.1.3.1	git-fetch method	13
2.1.3.2	Snippets	14
2.1.3.3	Inputs	14
2.1.3.4		15
2.1.3.5	Build system arguments	15
2.1.3.6	Code staging	17
2.1.3.7	Utility functions	17
2.1.3.8	Module prefix	18
2.1.4		18
2.1.5		18
2.1.5.1		18
2.1.5.2		20
2.1.5.3		20
2.1.6		20
2.1.7		21
2.1.8		21
2.2	Packaging Workflows	21
2.2.1	Packaging Rust Crates	21
2.2.1.1	Common Workflow for Rust Packaging	21
2.2.1.2	Cargo Workspaces and Development Snapshots	24
2.2.1.3	Using Rust Libraries in Other Build Systems	26
3		28
3.1	TTY	28
3.2		28
3.3	API Guix	32
3.4		35
3.4.1	Configuration for use as a two-factor authenticator (2FA)	35
3.4.2	Disabling OTP code generation for a Yubikey	36
3.4.3	Requiring a Yubikey to open a KeePassXC database	36
3.5	DNS mcron	37

3.6	Wireguard VPN	38
3.6.1	Using Wireguard tools	38
3.6.2	Using NetworkManager	38
3.7		39
3.7.1	StumpWM	39
3.7.2		40
3.7.2.1	Xorg	40
3.8	Guix Linode	40
3.9	Guix Kimsufi	44
3.10		48
3.11	Tor	48
3.12	NGINX Lua	49
3.13	Bluetooth-	50
4		55
4.1	Guix	55
4.2	Guix System	57
4.2.1		58
4.2.2		60
5		62
5.1	QEMU	62
5.1.1	Creating a network bridge interface	62
5.1.2	Configuring the QEMU bridge helper script	62
5.1.3	Invoking QEMU with the right command line options	63
5.1.4	Networking issues caused by Docker	63
5.2	libvirt	63
5.2.1	Creating a virtual network switch	64
5.2.2	Configuring the static routes for your virtual bridge	64
5.3	NAT libvirt	65
5.3.1	Creating the virtual network bridge	65
5.3.2	Running dnsmasq for the virtual network bridge	65
5.3.3	Configuring NAT for the virtual network switch	66
5.4		67
6		68
6.1	Guix	68
6.1.1		69
6.1.2		71
6.1.3		71
6.1.4		71
6.1.5		72

7		74
7.1		74
7.2	Level 1: Building with Guix	76
7.3	Level 2: The Repository as a Channel	77
7.4	Bonus: Package Variants	79
7.5	Level 3: Setting Up Continuous Integration	80
7.6	Bonus: Build manifest	81
7.7		83
8		85
8.1	Guix direnv	85
9	Using Guix for Reproducible Research	88
9.1	Step 1: Setting Up the Environment	88
9.2	Step 2: Recording the Environment	89
9.3	Step 3: Ensuring Long-Term Source Code Archiving	90
9.4	Step 4: Referencing the Software Environment	90
10	Installing Guix on a Cluster	92
10.1		92
10.2		93
10.3		95
10.4		96
10.5	Security Considerations	96
11	Guix	98
11.1	Postgres Cuirass	98
12		99
Appendix A	GNU	100
		108

1 Scheme

GNU Guix Scheme, . Scheme , , , ..
 Scheme Guix — !
 !

1.1 Scheme

Guix Guile Scheme. , `guix install guile` *REPL*— *read-eval-print loop* (<https://ru.wikipedia.org/wiki/REPL>)— `guile` .

, `guix shell guile -- guile`, Guile .

, `REPL`; , “ $\Rightarrow$ ”, , , “ $\vdash$ ”, . See Section “Using Guile Interactively” in *GNU Guile Reference Manual*, `REPL`.

- Scheme (*s-expression* Lisp lingo). , , , . `#true` `#false` (`#t` `#f`) “true” and “false” .

:

" !"

$\Rightarrow$ " !"

17

$\Rightarrow$ 17

(display (string-append " " "Guix" "\n"))

$\vdash$ Guix

$\Rightarrow$ `#<unspecified>`

- . , , .
- *procedures* Scheme — `lambda`:

(lambda (x) (* x x))

$\Rightarrow$ `#<procedure 120e348 at <unknown port>:24:0 (x)>`

. , `lambda` , :

((lambda (x) (* x x)) 3)

$\Rightarrow$ 9

- , , , .

- `define`:
 (define a 3)
 (define square (lambda (x) (* x x)))
 (square a)
 $\Rightarrow$ 9

- :
 (define (square x) (* x x))

- `list`:
 (list 2 a 5 7)
 $\Rightarrow$ (2 3 5 7)

- (srfi srfi-1) (see Section “list processing” in *GNU Guile Reference Manual*).
:

```
(use-modules (srfi srfi-1)) ;

(append (list 1 2) (list 3 4))
⇒ (1 2 3 4)

(map (lambda (x) (* x x)) (list 1 2 3 4))
⇒ (1 4 9 16)

(delete 3 (list 1 2 3 4))      ⇒ (1 2 4)
(filter odd? (list 1 2 3 4))   ⇒ (1 3)
(remove even? (list 1 2 3 4))  ⇒ (1 3)
(find number? (list "a" 42 "b")) ⇒ 42
```

, map, filter, remove find !

- The *quote* disables evaluation of a parenthesized expression, also called an S-expression or “s-exp”: the first term is not called over the other terms (see Section “Expression Syntax” in *GNU Guile Reference Manual*). Thus it effectively returns a list of terms.

```
'(display (string-append "Hello " "Guix" "\n"))
⇒ (display (string-append "Hello " "Guix" "\n"))
```

```
'(2 a 5 7)
⇒ (2 a 5 7)
```

- *quasiquote* (```, `)` , *unquote* (`,`, `)` . , , .
``(2 a 5 7 (2 ,a 5 ,(+ a 4)))`
`⇒ (2 a 5 7 (2 3 5 7))`

, - : , (a), .

- Guix S- " " G-expressions “gexps”, *quasiquote* *unquote*: `#~ ( gexp) #$( ungexp)`. *stage code for later execution*.

, gexps , . :

```
(use-modules (guix gexp) ;so we can write gexps
              (gnu packages base)) ;for 'coreutils'
```

```
;; Below is a G-expression representing staged code.
```

```
#~(begin
  ;; Invoke 'ls' from the package defined by the 'coreutils'
  ;; variable.
  (system* #$(file-append coreutils "/bin/ls") "-l")

  ;; Create this package's output directory.
  (mkdir #$output))
```

See Section “G-Expressions” in *GNU Guix Reference Manual*, for more on gexps.

- *let* (see Section “Local Bindings” in *GNU Guile Reference Manual*):
`(define x 10)`

```
(let ((x 2)
      (y 3))
  (list x y))
⇒ (2 3)
```

```
x
⇒ 10
```

```
y
[error] In procedure module-lookup: Unbound variable: y
```

Use `let*` to allow later variable declarations to refer to earlier definitions.

```
(let* ((x 2)
       (y (* x 3)))
  (list x y))
⇒ (2 6)
```

- *Keywords* are typically used to identify the named parameters of a procedure. They are prefixed by `#:` (hash, colon) followed by alphanumeric characters: `#:like-this`. See Section “Keywords” in *GNU Guile Reference Manual*.
- The percentage `%` is typically used for read-only global variables in the build stage. Note that it is merely a convention, like `_` in C. Scheme treats `%` exactly the same as any other letter.
- Modules are created with `define-module` (see Section “Creating Guile Modules” in *GNU Guile Reference Manual*). For instance

```
(define-module (guix build-system ruby)
  #:use-module (guix store)
  #:export (ruby-build
            ruby-build-system))
```

defines the module `guix build-system ruby` which must be located in `guix/build-system/ruby.scm` somewhere in the Guile load path. It depends on the `(guix store)` module and it exports two variables, `ruby-build` and `ruby-build-system`.

See Section “Package Modules” in *GNU Guix Reference Manual*, for info on modules that define packages.

: Scheme is a language that has been widely used to teach programming and you’ll find plenty of material using it as a vehicle. Here’s a selection of documents to learn more about Scheme:

- *A Scheme Primer* (<https://spritely.institute/static/papers/scheme-primer.html>), by Christine Lemmer-Webber and the Spritely Institute.
- *Structure and Interpretation of Computer Programs* (<https://sarabander.github.io/sicp/>), by Harold Abelson and Gerald Jay Sussman, with Julie Sussman. Colloquially known as “SICP”, this book is a reference.

You can also install it and read it from your computer:

```
guix install book-sicp info-reader
```


`info sicp`

You'll find more books, tutorials and other resources at <https://schemers.org/>.

2

This chapter is dedicated to teaching you how to add packages to the collection of packages that come with GNU Guix. This involves writing package definitions in Guile Scheme, organizing them in package modules, and building them.

2.1

GNU Guix stands out as the *hackable* package manager, mostly because it uses GNU Guile (<https://www.gnu.org/software/guile/>), a powerful high-level programming language, one of the Scheme (https://en.wikipedia.org/wiki/Scheme_%28programming_language%29) dialects from the Lisp family (https://en.wikipedia.org/wiki/Lisp_%28programming_language%29).

Package definitions are also written in Scheme, which empowers Guix in some very unique ways, unlike most other package managers that use shell scripts or simple languages.

- Use functions, structures, macros and all of Scheme expressiveness for your package definitions.
- Inheritance makes it easy to customize a package by inheriting from it and modifying only what is needed.
- Batch processing: the whole package collection can be parsed, filtered and processed. Building a headless server with all graphical interfaces stripped out? It's possible. Want to rebuild everything from source using specific compiler optimization flags? Pass the `#:make-flags "..."` argument to the list of packages. It wouldn't be a stretch to think Gentoo USE flags (https://wiki.gentoo.org/wiki/USE_flag) here, but this goes even further: the changes don't have to be thought out beforehand by the packager, they can be *programmed* by the user!

The following tutorial covers all the basics around package creation with Guix. It does not assume much knowledge of the Guix system nor of the Lisp language. The reader is only expected to be familiar with the command line and to have some basic programming knowledge.

2.1.1 “Hello World”

The “Defining Packages” section of the manual introduces the basics of Guix packaging (see Section “Defining Packages” in *GNU Guix Reference Manual*). In the following section, we will partly go over those basics again.

GNU Hello is a dummy project that serves as an idiomatic example for packaging. It uses the GNU build system (`./configure && make && make install`). Guix already provides a package definition which is a perfect example to start with. You can look up its declaration with `guix edit hello` from the command line. Let's see how it looks:

```
(define-public hello
  (package
    (name "hello")
    (version "2.10")
    (source (origin
              (method url-fetch)
```

```

(uri (string-append "mirror://gnu/hello/hello-" version
                    ".tar.gz"))
(sha256
 (base32
  "0ssi1wpaf7plawqqjwigppsg5fyh99vdlb9kz17c9lng89ndq1i"))))
(build-system gnu-build-system)
(synopsis "Hello, GNU world: An example GNU package")
(description
 "GNU Hello prints the message \"Hello, world!\" and then exits. It
 serves as an example of standard GNU coding practices. As such, it supports
 command-line arguments, multiple languages, and so on.")
(home-page "https://www.gnu.org/software/hello/")
(license gpl3+))

```

As you can see, most of it is rather straightforward. But let's review the fields together:

- 'name' The project name. Using Scheme conventions, we prefer to keep it lower case, without underscore and using dash-separated words.
- 'source' This field contains a description of the source code origin. The `origin` record contains these fields:
 1. The method, here `url-fetch` to download via HTTP/FTP, but other methods exist, such as `git-fetch` for Git repositories.
 2. The URI, which is typically some `https://` location for `url-fetch`. Here the special 'mirror://gnu' refers to a set of well known locations, all of which can be used by Guix to fetch the source, should some of them fail.
 3. The `sha256` checksum of the requested file. This is essential to ensure the source is not corrupted. Note that Guix works with base32 strings, hence the call to the `base32` function.
- 'build-system' This is where the power of abstraction provided by the Scheme language really shines: in this case, the `gnu-build-system` abstracts away the famous `./configure && make && make install` shell invocations. Other build systems include the `trivial-build-system` which does not do anything and requires from the packager to program all the build steps, the `python-build-system`, the `emacs-build-system`, and many more (see Section "Build Systems" in *GNU Guix Reference Manual*).
- 'synopsis' It should be a concise summary of what the package does. For many packages a tagline from the project's home page can be used as the synopsis.
- 'description' Same as for the synopsis, it's fine to re-use the project description from the homepage. Note that Guix uses Texinfo syntax.
- 'home-page' Use HTTPS if available.
- 'license' See `guix/licenses.scm` in the project source for a full list of available licenses.

Time to build our first package! Nothing fancy here for now: we will stick to a dummy `my-hello`, a copy of the above declaration.

As with the ritualistic “Hello World” taught with most programming languages, this will possibly be the most “manual” approach. We will work out an ideal setup later; for now we will go the simplest route.

Save the following to a file `my-hello.scm`.

```
(use-modules (guix packages)
             (guix download)
             (guix build-system gnu)
             (guix licenses))

(package
  (name "my-hello")
  (version "2.10")
  (source (origin
    (method url-fetch)
    (uri (string-append "mirror://gnu/hello/hello-" version
                        ".tar.gz"))
    (sha256
      (base32
        "0ssi1wpaf7plaswqqjwigppsg5fyh99vdlb9kzl7c9lng89ndq1i"))))
  (build-system gnu-build-system)
  (synopsis "Hello, Guix world: An example custom Guix package")
  (description
    "GNU Hello prints the message \"Hello, world!\" and then exits. It
    serves as an example of standard GNU coding practices. As such, it supports
    command-line arguments, multiple languages, and so on.")
  (home-page "https://www.gnu.org/software/hello/")
  (license gpl3+))
```

We will explain the extra code in a moment.

Feel free to play with the different values of the various fields. If you change the source, you’ll need to update the checksum. Indeed, Guix refuses to build anything if the given checksum does not match the computed checksum of the source code. To obtain the correct checksum of the package declaration, we need to download the source, compute the sha256 checksum and convert it to base32.

Thankfully, Guix can automate this task for us; all we need is to provide the URI:

```
$ guix download mirror://gnu/hello/hello-2.10.tar.gz
```

```
Starting download of /tmp/guix-file.JLYgL7
```

```
From https://ftpmirror.gnu.org/gnu/hello/hello-2.10.tar.gz...
```

```
following redirection to `https://mirror.ibcp.fr/pub/gnu/hello/hello-2.10.tar.gz'...
```

```
...10.tar.gz 709KiB 2.5MiB/s 00:00 [#####]
```

```
/gnu/store/hbdalsf5lpf01x4dcknwx6xbn6n5km6k-hello-2.10.tar.gz
```

```
0ssi1wpaf7plaswqqjwigppsg5fyh99vdlb9kzl7c9lng89ndq1i
```

In this specific case the output tells us which mirror was chosen. If the result of the above command is not the same as in the above snippet, update your `my-hello` declaration accordingly.

Note that GNU package tarballs come with an OpenPGP signature, so you should definitely check the signature of this tarball with ‘gpg’ to authenticate it before going further:

```
$ guix download mirror://gnu/hello/hello-2.10.tar.gz.sig
```

```
Starting download of /tmp/guix-file.03tFfb
```

```
From https://ftpmirror.gnu.org/gnu/hello/hello-2.10.tar.gz.sig...
```

```
following redirection to `https://ftp.igh.cnrs.fr/pub/gnu/hello/hello-2.10.tar.gz.sig'
```

```
....tar.gz.sig 819B
```

```
/gnu/store/rzs8wba9ka7grrmgcpfyxvs58mly0sx6-hello-2.10.tar.gz.sig
```

```
0q0v86n3y38z17rl146gdakw9xc4mcscpk8dscs412j22glrv9jf
```

```
$ gpg --verify /gnu/store/rzs8wba9ka7grrmgcpfyxvs58mly0sx6-hello-2.10.tar.gz.sig /gnu/
```

```
gpg: Signature made Sun 16 Nov 2014 01:08:37 PM CET
```

```
gpg: using RSA key A9553245FDE9B739
```

```
gpg: Good signature from "Sami Kerola <kerolasa@iki.fi>" [unknown]
```

```
gpg: aka "Sami Kerola (http://www.iki.fi/kerolasa/) <kerolasa@iki.fi>"
```

```
gpg: WARNING: This key is not certified with a trusted signature!
```

```
gpg: There is no indication that the signature belongs to the owner.■
```

```
Primary key fingerprint: 8ED3 96E3 7E38 D471 A005 30D3 A955 3245 FDE9 B739■
```

You can then happily run

```
$ guix package --install-from-file=my-hello.scm
```

You should now have `my-hello` in your profile!

```
$ guix package --list-installed=my-hello
```

```
my-hello 2.10 out
```

```
/gnu/store/f1db2mfm8syb8qvc357c53slbv1g9m9-my-hello-2.10
```

We’ve gone as far as we could without any knowledge of Scheme. Before moving on to more complex packages, now is the right time to brush up on your Scheme knowledge. See Section 1.1 [Scheme], page 1, to get up to speed.

2.1.2

In the rest of this chapter we will rely on some basic Scheme programming knowledge. Now let’s detail the different possible setups for working on Guix packages.

There are several ways to set up a Guix packaging environment.

We recommend you work directly on the Guix source checkout since it makes it easier for everyone to contribute to the project.

But first, let’s look at other possibilities.

2.1.2.1

This is what we previously did with ‘`my-hello`’. With the Scheme basics we’ve covered, we are now able to explain the leading chunks. As stated in `guix package --help`:

```
-f, --install-from-file=FILE
```

```
install the package that the code within FILE
```

evaluates to

Thus the last expression *must* return a package, which is the case in our earlier example.

The `use-modules` expression tells which of the modules we need in the file. Modules are a collection of values and procedures. They are commonly called “libraries” or “packages” in other programming languages.

2.1.2.2

Guix and its package collection can be extended through *channels*. A channel is a Git repository, public or not, containing `.scm` files that provide packages (see Section “Defining Packages” in *GNU Guix Reference Manual*) or services (see Section “Defining Services” in *GNU Guix Reference Manual*).

How would you go about creating a channel? First, create a directory that will contain your `.scm` files, say `~/my-channel`:

```
mkdir ~/my-channel
```

Suppose you want to add the ‘my-hello’ package we saw previously; it first needs some adjustments:

```
(define-module (my-hello)
  #:use-module (guix licenses)
  #:use-module (guix packages)
  #:use-module (guix build-system gnu)
  #:use-module (guix download))

(define-public my-hello
  (package
    (name "my-hello")
    (version "2.10")
    (source (origin
              (method url-fetch)
              (uri (string-append "mirror://gnu/hello/hello-" version
                                   ".tar.gz"))
              (sha256
               (base32
                "0ssi1wpaf7plawqqjwigppsg5fyh99vdlb9kzl7c9lng89ndqi"))))
    (build-system gnu-build-system)
    (synopsis "Hello, Guix world: An example custom Guix package")
    (description
     "GNU Hello prints the message \"Hello, world!\" and then exits. It
     serves as an example of standard GNU coding practices. As such, it supports
     command-line arguments, multiple languages, and so on.")
    (home-page "https://www.gnu.org/software/hello/")
    (license gpl3+)))
```

Note that we have assigned the package value to an exported variable name with `define-public`. This is effectively assigning the package to the `my-hello` variable so that it can be referenced, among other as dependency of other packages.

If you use `guix package --install-from-file=my-hello.scm` on the above file, it will fail because the last expression, `define-public`, does not return a package. If you want to use `define-public` in this use-case nonetheless, make sure the file ends with an evaluation of `my-hello`:

```
;; ...
(define-public my-hello
  ;; ...
)
```

`my-hello`

This last example is not very typical.

Now how do you make that package visible to `guix` commands so you can test your packages? You need to add the directory to the search path using the `-L` command-line option, as in these examples:

```
guix show -L ~/my-channel my-hello
guix build -L ~/my-channel my-hello
```

The final step is to turn `~/my-channel` into an actual channel, making your package collection seamlessly available *via* any `guix` command. To do that, you first need to make it a Git repository:

```
cd ~/my-channel
git init
git add my-hello.scm
git commit -m "First commit of my channel."
```

And that's it, you have a channel! From there on, you can add this channel to your channel configuration in `~/.config/guix/channels.scm` (see Section “Specifying Additional Channels” in *GNU Guix Reference Manual*); assuming you keep your channel local for now, the `channels.scm` would look something like this:

```
(append (list (channel
  (name 'my-channel)
  (url (string-append "file://" (getenv "HOME")
    "/my-channel"))))
  %default-channels)
```

Next time you run `guix pull`, your channel will be picked up and the packages it defines will be readily available to all the `guix` commands, even if you do not pass `-L`. The `guix describe` command will show that Guix is, indeed, using both the `my-channel` and the `guix` channels.

See Section “Creating a Channel” in *GNU Guix Reference Manual*, for details.

2.1.2.3

Working directly on the Guix project is recommended: it reduces the friction when the time comes to submit your changes upstream to let the community benefit from your hard work!

Unlike most software distributions, the Guix repository holds in one place both the tooling (including the package manager) and the package definitions. This choice was made so that it would give developers the flexibility to modify the API without breakage by updating all packages at the same time. This reduces development inertia.

Check out the official Git (<https://git-scm.com/>) repository:

```
$ git clone https://git.guix.gnu.org/guix.git
```

In the rest of this article, we use ‘\$GUIX_CHECKOUT’ to refer to the location of the checkout.

Follow the instructions in the manual (see Section “Contributing” in *GNU Guix Reference Manual*) to set up the repository environment.

Once ready, you should be able to use the package definitions from the repository environment.

Feel free to edit package definitions found in ‘\$GUIX_CHECKOUT/gnu/packages’.

The ‘\$GUIX_CHECKOUT/pre-inst-env’ script lets you use ‘guix’ over the package collection of the repository (see Section “Running Guix Before It Is Installed” in *GNU Guix Reference Manual*).

- Search packages, such as Ruby:

```
$ cd $GUIX_CHECKOUT
$ ./pre-inst-env guix package --list-available=ruby
ruby      1.8.7-p374      out      gnu/packages/ruby.scm:119:2
ruby      2.1.6          out      gnu/packages/ruby.scm:91:2
ruby      2.2.2          out      gnu/packages/ruby.scm:39:2
```

- Build a package, here Ruby version 2.1:

```
$ ./pre-inst-env guix build --keep-failed ruby@2.1
/gnu/store/c13v73jxmj2nir2xjqaz5259zywsa9zi-ruby-2.1.6
```

- Install it to your user profile:

```
$ ./pre-inst-env guix package --install ruby@2.1
```

- Check for common mistakes:

```
$ ./pre-inst-env guix lint ruby@2.1
```

Guix strives at maintaining a high packaging standard; when contributing to the Guix project, remember to

- follow the coding style (see Section “Coding Style” in *GNU Guix Reference Manual*),
- and review the check list from the manual (see Section “Submitting Patches” in *GNU Guix Reference Manual*).

Once you are happy with the result, you are welcome to send your contribution to make it part of Guix. This process is also detailed in the manual. (see Section “Contributing” in *GNU Guix Reference Manual*)

It’s a community effort so the more join in, the better Guix becomes!

2.1.3

The above “Hello World” example is as simple as it goes. Packages can be more complex than that and Guix can handle more advanced scenarios. Let’s look at another, more sophisticated package (slightly modified from the source):

```
(define-module (gnu packages version-control)
  #:use-module ((guix licenses) #:prefix license:)
  #:use-module (guix utils))
```



```

#:use-module (guix packages)
#:use-module (guix git-download)
#:use-module (guix build-system cmake)
#:use-module (gnu packages)
#:use-module (gnu packages compression)
#:use-module (gnu packages pkg-config)
#:use-module (gnu packages python)
#:use-module (gnu packages ssh)
#:use-module (gnu packages tls)
#:use-module (gnu packages web))

(define-public my-libgit2
  (let ((commit "e98d0a37c93574d2c6107bf7f31140b548c6a7bf")
        (revision "1"))
    (package
      (name "my-libgit2")
      (version (git-version "0.26.6" revision commit))
      (source (origin
        (method git-fetch)
        (uri (git-reference
          (url "https://github.com/libgit2/libgit2/")
          (commit commit)))
        (file-name (git-file-name name version))
        (sha256
          (base32
            "17pjvprmdrx4h6bb1hhc98w9qi6ki7yl57f090n9kbhswxqfs7s3"))
        (patches (search-patches "libgit2-mtime-0.patch"))
        (modules '((guix build utils)))
        ;; Remove bundled software.
        (snippet '(delete-file-recursively "deps"))))
      (build-system cmake-build-system)
      (outputs '("out" "debug"))
      (arguments
        `(#:tests? #true ; Run the test suite (this is the default)
          #:configure-flags '("-DUSE_SHA1DC=ON") ; SHA-1 collision detection
          #:phases
          (modify-phases %standard-phases
            (add-after 'unpack 'fix-hardcoded-paths
              (lambda _
                (substitute* "tests/repo/init.c"
                  ((#!/bin/sh) (string-append "#!" (which "sh"))))
                (substitute* "tests/clar/fs.h"
                  ((/bin/cp) (which "cp"))
                  ((/bin/rm) (which "rm")))))
              ;; Run checks more verbosely.
              (replace 'check
                (lambda* (#:key tests? #:allow-other-keys)

```

```

        (when tests?
          (invoke "./libgit2_clar" "-v" "-Q"))))
      (add-after 'unpack 'make-files-writable-for-tests
        (lambda _ (for-each make-file-writable (find-files ".")))))■
(inputs
 (list libssh2 http-parser python-wrapper))
(native-inputs
 (list pkg-config))
(propagated-inputs
 ;; These two libraries are in 'Requires.private' in libgit2.pc.
 (list openssl zlib))
(home-page "https://libgit2.github.com/")
(synopsis "Library providing Git core methods")
(description
 "Libgit2 is a portable, pure C implementation of the Git core methods■
 provided as a re-entrant linkable library with a solid API, allowing you to■
 write native speed custom Git applications in any language with bindings.")■
;; GPLv2 with linking exception
(license license:gpl2)))

```

(In those cases where you only want to tweak a few fields from a package definition, you should rely on inheritance instead of copy-pasting everything. See below.)

Let's discuss those fields in depth.

2.1.3.1 git-fetch method

Unlike the `url-fetch` method, `git-fetch` expects a `git-reference` which takes a Git repository and a commit. The commit can be any Git reference such as tags, so if the `version` is tagged, then it can be used directly. Sometimes the tag is prefixed with a `v`, in which case you'd use `(commit (string-append "v" version))`.

To ensure that the source code from the Git repository is stored in a directory with a descriptive name, we use `(file-name (git-file-name name version))`.

The `git-version` procedure can be used to derive the version when packaging programs for a specific commit, following the Guix contributor guidelines (see Section “Version Numbers” in *GNU Guix Reference Manual*).

How does one obtain the `sha256` hash that's in there, you ask? By invoking `guix hash` on a checkout of the desired commit, along these lines:

```

git clone https://github.com/libgit2/libgit2/
cd libgit2
git checkout v0.26.6
guix hash -rx .

```

`guix hash -rx` computes a SHA256 hash over the whole directory, excluding the `.git` sub-directory (see Section “Invoking `guix hash`” in *GNU Guix Reference Manual*).

In the future, `guix download` will hopefully be able to do these steps for you, just like it does for regular downloads.

2.1.3.2 Snippets

Snippets are quoted (i.e. non-evaluated) Scheme code that are a means of patching the source. They are a Guix-y alternative to the traditional `.patch` files. Because of the quote, the code is only evaluated when passed to the Guix daemon for building. There can be as many snippets as needed.

Snippets might need additional Guile modules which can be imported from the `modules` field.

2.1.3.3 Inputs

There are 3 different input types. In short:

native-inputs

Required for building but not runtime – installing a package through a substitute won't install these inputs.

inputs

Installed in the store but not in the profile, as well as being present at build time.

propagated-inputs

Installed in the store and in the profile, as well as being present at build time.

See Section “package Reference” in *GNU Guix Reference Manual* for more details.

The distinction between the various inputs is important: if a dependency can be handled as an *input* instead of a *propagated input*, it should be done so, or else it “pollutes” the user profile for no good reason.

For instance, a user installing a graphical program that depends on a command line tool might only be interested in the graphical part, so there is no need to force the command line tool into the user profile. The dependency is a concern to the package, not to the user. *Inputs* make it possible to handle dependencies without bugging the user by adding undesired executable files (or libraries) to their profile.

Same goes for *native-inputs*: once the program is installed, build-time dependencies can be safely garbage-collected. It also matters when a substitute is available, in which case only the *inputs* and *propagated inputs* will be fetched: the *native inputs* are not required to install a package from a substitute.

: You may see here and there snippets where package inputs are written quite differently, like so:

```
;; The "old style" for inputs.
(inputs
  `(("libssh2" ,libssh2)
     ("http-parser" ,http-parser)
     ("python" ,python-wrapper)))
```

This is the “old style”, where each input in the list is explicitly given a label (a string). It is still supported but we recommend using the style above instead. See Section “package Reference” in *GNU Guix Reference Manual*, for more info.

2.1.3.4

Just like how a package can have multiple inputs, it can also produce multiple outputs.

Each output corresponds to a separate directory in the store.

The user can choose which output to install; this is useful to save space or to avoid polluting the user profile with unwanted executables or libraries.

Output separation is optional. When the `outputs` field is left out, the default and only output (the complete package) is referred to as `"out"`.

Typical separate output names include `debug` and `doc`.

It's advised to separate outputs only when you've shown it's worth it: if the output size is significant (compare with `guix size`) or in case the package is modular.

2.1.3.5 Build system arguments

The `arguments` is a keyword-value list used to configure the build process.

The simplest argument `#:tests?` can be used to disable the test suite when building the package. This is mostly useful when the package does not feature any test suite. It's strongly recommended to keep the test suite on if there is one.

Another common argument is `:make-flags`, which specifies a list of flags to append when running `make`, as you would from the command line. For instance, the following flags

```
#:make-flags (list (string-append "prefix=" (assoc-ref %outputs "out"))
                  "CC=gcc")
```

translate into

```
$ make CC=gcc prefix=/gnu/store/...-<out>
```

This sets the C compiler to `gcc` and the `prefix` variable (the installation directory in Make parlance) to `(assoc-ref %outputs "out")`, which is a build-stage global variable pointing to the destination directory in the store (something like `/gnu/store/...-my-libgit2-20180408`).

Similarly, it's possible to set the configure flags:

```
#:configure-flags '("-DUSE_SHA1DC=ON")
```

The `%build-inputs` variable is also generated in scope. It's an association table that maps the input names to their store directories.

The `phases` keyword lists the sequential steps of the build system. Typically phases include `unpack`, `configure`, `build`, `install` and `check`. To know more about those phases, you need to work out the appropriate build system definition in `'$GUIX_CHECKOUT/guix/build/gnu-build-system.scm'`:

```
(define %standard-phases
  ;; Standard build phases, as a list of symbol/procedure pairs.
  (let-syntax ((phases (syntax-rules ()
                        ((_ p ...) `((p . ,p) ...))))
    (phases set-SOURCE-DATE-EPOCH set-paths install-locale unpack
            bootstrap
            patch-usr-bin-file
            patch-source-shebangs configure patch-generated-file-shebangs
            build check install
```

```

patch-shebangs strip
validate-runpath
validate-documentation-location
delete-info-dir-file
patch-dot-desktop-files
make-dynamic-linker-cache
install-license-files
reset-gzip-timestamps
compress-documentation)))

```

Or from the REPL:

```

(add-to-load-path "/path/to/guix/checkout")
,use (guix build gnu-build-system)
(map car %standard-phases)
⇒ (set-SOURCE-DATE-EPOCH set-paths install-locale unpack bootstrap patch-usr-bin-file

```

If you want to know more about what happens during those phases, consult the associated procedures.

For instance, as of this writing the definition of `unpack` for the GNU build system is:

```

(define* (unpack #:key source #:allow-other-keys)
  "Unpack SOURCE in the working directory, and change directory within the
  source. When SOURCE is a directory, copy it in a sub-directory of the current
  working directory."
  (if (file-is-directory? source)
      (begin
        (mkdir "source")
        (chdir "source")

        ;; Preserve timestamps (set to the Epoch) on the copied tree so that
        ;; things work deterministically.
        (copy-recursively source ".")
        #:keep-mtime? #t)
      ;; Make the source checkout files writable, for convenience.
      (for-each (lambda (f)
                  (false-if-exception (make-file-writable f)))
                (find-files ".")))
    (begin
      (cond
        ((string-suffix? ".zip" source)
         (invoke "unzip" source))
        ((tarball? source)
         (invoke "tar" "xvf" source))
        (else
         (let ((name (strip-store-file-name source))
               (command (compressor source)))
           (copy-file source name)
           (when command
            (invoke command "--decompress" name))))))

```

```
;; Attempt to change into child directory.
(and=> (first-subdirectory ".") chdir))))
```

Note the `chdir` call: it changes the working directory to where the source was unpacked. Thus every phase following the `unpack` will use the source as a working directory, which is why we can directly work on the source files. That is to say, unless a later phase changes the working directory to something else.

We modify the list of `%standard-phases` of the build system with the `modify-phases` macro as per the list of specified modifications, which may have the following forms:

- `(add-before phase new-phase procedure)`: Run *procedure* named *new-phase* before *phase*.
- `(add-after phase new-phase procedure)`: Same, but afterwards.
- `(replace phase procedure)`.
- `(delete phase)`.

The *procedure* supports the keyword arguments `inputs` and `outputs`. Each input (whether *native*, *propagated* or not) and output directory is referenced by their name in those variables. Thus `(assoc-ref outputs "out")` is the store directory of the main output of the package. A phase procedure may look like this:

```
(lambda* (#:key inputs outputs #:allow-other-keys)
  (let ((bash-directory (assoc-ref inputs "bash"))
        (output-directory (assoc-ref outputs "out"))
        (doc-directory (assoc-ref outputs "doc")))
    ;; ... ))
```

Its return value is ignored.

2.1.3.6 Code staging

The astute reader may have noticed the quasi-quote and comma syntax in the argument field. Indeed, the build code in the package declaration should not be evaluated on the client side, but only when passed to the Guix daemon. This mechanism of passing code around two running processes is called code staging (<https://arxiv.org/abs/1709.00833>).

2.1.3.7 Utility functions

When customizing `phases`, we often need to write code that mimics the equivalent system invocations (`make`, `mkdir`, `cp`, etc.) commonly used during regular “Unix-style” installations.

Some like `chmod` are native to Guile. See *Guile reference manual* for a complete list.

Guix provides additional helper functions which prove especially handy in the context of package management.

Some of those functions can be found in ‘`$GUIX_CHECKOUT/guix/guix/build/utils.scm`’. Most of them mirror the behaviour of the traditional Unix system commands:

which Like the ‘`which`’ system command.

find-files Akin to the ‘`find`’ system command.

mkdir-p Like ‘`mkdir -p`’, which creates all parents as needed.

install-file

Similar to ‘install’ when installing a file to a (possibly non-existing) directory. Guile has `copy-file` which works like ‘cp’.

copy-recursively

Like ‘cp -r’.

delete-file-recursively

Like ‘rm -rf’.

invoke Run an executable. This should be used instead of `system*`.

with-directory-excursion

Run the body in a different working directory, then restore the previous working directory.

substitute*

A “sed-like” function.

See Section “Build Utilities” in *GNU Guix Reference Manual*, for more information on these utilities.

2.1.3.8 Module prefix

The `license` in our last example needs a prefix: this is because of how the `license` module was imported in the package, as `#:use-module ((guix licenses) #:prefix license:)`. The Guile module import mechanism (see Section “Using Guile Modules” in *Guile reference manual*) gives the user full control over namespacing: this is needed to avoid clashes between, say, the ‘zlib’ variable from ‘`licenses.scm`’ (a *license* value) and the ‘zlib’ variable from ‘`compression.scm`’ (a *package* value).

2.1.4

What we’ve seen so far covers the majority of packages using a build system other than the `trivial-build-system`. The latter does not automate anything and leaves you to build everything manually. This can be more demanding and we won’t cover it here for now, but thankfully it is rarely necessary to fall back on this system.

For the other build systems, such as ASDF, Emacs, Perl, Ruby and many more, the process is very similar to the GNU build system except for a few specialized arguments.

See Section “Build Systems” in *GNU Guix Reference Manual*, for more information on build systems, or check the source code in the ‘`$GUIX_CHECKOUT/guix/build`’ and ‘`$GUIX_CHECKOUT/guix/build-system`’ directories.

2.1.5

We can’t repeat it enough: having a full-fledged programming language at hand empowers us in ways that reach far beyond traditional package management.

Let’s illustrate this with some awesome features of Guix!

2.1.5.1

You might find some build systems good enough that there is little to do at all to write a package, to the point that it becomes repetitive and tedious after a while. A *raison d’être* of

computers is to replace human beings at those boring tasks. So let's tell Guix to do this for us and create the package definition of an R package from CRAN (the output is trimmed for conciseness):

```
$ guix import cran --recursive walrus

(define-public r-mc2d
  ; ...
  (license gpl2+)))

(define-public r-jmvcore
  ; ...
  (license gpl2+)))

(define-public r-wrs2
  ; ...
  (license gpl3)))

(define-public r-walrus
  (package
    (name "r-walrus")
    (version "1.0.3")
    (source
      (origin
        (method url-fetch)
        (uri (cran-uri "walrus" version))
        (sha256
          (base32
            "1nk2glcvy4hyksl5ipq2mz8jy4fss90hx6cq98m3w96kzjni6jjj")))))
    (build-system r-build-system)
    (propagated-inputs
      (list r-ggplot2 r-jmvcore r-r6 r-wrs2))
    (home-page "https://github.com/jamovi/walrus")
    (synopsis "Robust Statistical Methods")
    (description
      "This package provides a toolbox of common robust statistical
      tests, including robust descriptives, robust t-tests, and robust ANOVA.
      It is also available as a module for 'jamovi' (see
      <https://www.jamovi.org> for more information). Walrus is based on the
      WRS2 package by Patrick Mair, which is in turn based on the scripts and
      work of Rand Wilcox. These analyses are described in depth in the book
      'Introduction to Robust Estimation & Hypothesis Testing'.")
    (license gpl3)))
```

The recursive importer won't import packages for which Guix already has package definitions, except for the very first.

Not all applications can be packaged this way, only those relying on a select number of supported systems. Read about the full list of importers in the `guix` import section of the manual (see Section “Invoking `guix` import” in *GNU Guix Reference Manual*).

2.1.5.2

`Guix` can be smart enough to check for updates on systems it knows. It can report outdated package definitions with

```
$ guix refresh hello
```

In most cases, updating a package to a newer version requires little more than changing the version number and the checksum. `Guix` can do that automatically as well:

```
$ guix refresh hello --update
```

2.1.5.3

If you’ve started browsing the existing package definitions, you might have noticed that a significant number of them have a `inherit` field:

```
(define-public adwaita-icon-theme
  (package (inherit gnome-icon-theme)
    (name "adwaita-icon-theme")
    (version "3.26.1")
    (source (origin
      (method url-fetch)
      (uri (string-append "mirror://gnome/sources/" name "/"
        (version-major+minor version) "/"
        name "-" version ".tar.xz"))
      (sha256
        (base32
          "17fpahgh5dyckgz7rwqvzgnhx53cx9kr2xw0szprc6bnqy977fi8")))))
  (native-inputs (list `(.gtk+ "bin")))))
```

All unspecified fields are inherited from the parent package. This is very convenient to create alternative packages, for instance with different source, version or compilation options.

2.1.6

Sadly, some applications can be tough to package. Sometimes they need a patch to work with the non-standard file system hierarchy enforced by the store. Sometimes the tests won’t run properly. (They can be skipped but this is not recommended.) Other times the resulting package won’t be reproducible.

Should you be stuck, unable to figure out how to fix any sort of packaging issue, don’t hesitate to ask the community for help.

See the `Guix` homepage (<https://guix.gnu.org/contact/>) for information on the mailing lists, IRC, etc.

2.1.7

This tutorial was a showcase of the sophisticated package management that Guix boasts. At this point we have mostly restricted this introduction to the `gnu-build-system` which is a core abstraction layer on which more advanced abstractions are based.

Where do we go from here? Next we ought to dissect the innards of the build system by removing all abstractions, using the `trivial-build-system`: this should give us a thorough understanding of the process before investigating some more advanced packaging techniques and edge cases.

Other features worth exploring are the interactive editing and debugging capabilities of Guix provided by the Guile REPL.

Those fancy features are completely optional and can wait; now is a good time to take a well-deserved break. With what we’ve introduced here you should be well armed to package lots of programs. You can get started right away and hopefully we will see your contributions soon!

2.1.8

- The package reference in the manual (https://guix.gnu.org/manual/en/html_node/Defining-Packages.html)
- Pjotr’s hacking guide to GNU Guix (<https://gitlab.com/pjotrp/guix-notes/blob/master/HACKING.org>)
- “GNU Guix: Package without a scheme!” (<https://guix.gnu.org/guix-ghm-andreas-20130823.pdf>), by Andreas Enge

2.2 Packaging Workflows

The following sections provide real-life examples on working with specific build systems, serving as extensions to the concise packaging guidelines (see Section “Packaging Guidelines” in *GNU Guix Reference Manual*).

2.2.1 Packaging Rust Crates

In preparation, add the following packages to our environment:

```
$ guix shell rust rust:cargo cargo-audit cargo-license
```

2.2.1.1 Common Workflow for Rust Packaging

In this example, we’ll package `cargo-audit`, which is published on the crates.io (<https://crates.io>) Rust package repository. All its dependencies are on crates.io as well.

1. Since `cargo-audit` is available on crates.io, we can generate a template via the crates.io importer (see Section “Invoking guix import” in *GNU Guix Reference Manual*):

```
$ guix import crate cargo-audit
```

After manual editing, we’ll have the following definition:

```
(define-public cargo-audit
  (package
    (name "cargo-audit")
    (version "0.21.2")
```

```
(source
  (origin
    (method url-fetch)
    (uri (crate-uri "cargo-audit" version))
    (file-name (string-append name "-" version ".tar.gz"))
    (sha256
      (base32 "1a00yqpckkw86zh2hg7ra82c5fx0ird5766dyynimbvqiwg2ps0n"))))
  (build-system cargo-build-system)
  (arguments (list #:install-source? #f))
  (inputs (cargo-inputs 'cargo-audit))
  (home-page "https://rustsec.org/")
  (synopsis "Audit Cargo.lock for crates with security vulnerabilities")
  (description
    "This package provides a Cargo subcommand, @command{cargo audit}, to
    audit @file{Cargo.lock} for crates with security vulnerabilities.")
  (license (list license:asl2.0 license:expat))))
```

The identifier used to invoke `cargo-inputs`, in this case `'cargo-audit`, must be unique, usually matching the variable name of the package.

2. Unpack package source and navigate to the unpacked directory, then run the following commands:

```
$ cargo generate-lockfile
$ cargo audit
$ cargo license
```

`cargo generate-lockfile` updates dependencies to compatible versions. Applying it to all Rust applications helps reduce a great number of Rust libraries we need to check later. Although sometimes libraries may fail to follow the semantic versioning (<https://semver.org/>) scheme, it's still acceptable.

`cargo audit` checks known vulnerabilities and `cargo license` checks licenses, for all the dependencies. We must have an acceptable output of `cargo audit` and ensure all dependencies are licensed with our supported licenses (see Section “Defining Packages” in *GNU Guix Reference Manual*).

3. Import dependencies from the generated lockfile:

```
$ guix import --insert=gnu/packages/rust-crates.scm \
  crate --lockfile=/path/to/Cargo.lock cargo-audit

# Or use short options, in this case the shell processes file names
# before passing them to Guix, allowing tilde expansion, for example.
$ guix import -i gnu/packages/rust-crates.scm \
  crate -f /path/to/Cargo.lock cargo-audit
```

`cargo-audit` here must be consistent with the identifier used for `cargo-inputs` invocation in the package definition.

At this stage, the package `cargo-audit` is buildable.

4. Finally we'll unbundle the vendored dependencies. The lockfile importer inserts **TODO REVIEW**: comments for libraries with high probability of bundled dependencies.

`cargo-build-system` also performs additional check for binary files in its `check-for-pregenerated-files` phase, which usually indicates bundling:

```
$ ./pre-inst-env guix build cargo-audit
...
starting phase `check-for-pregenerated-files'
Searching for binary files...
./guix-vendor/rust-async-compression-0.4.21.tar.gz/tests/artifacts/dictionary-rust-
./guix-vendor/rust-async-compression-0.4.21.tar.gz/tests/artifacts/dictionary-rust-
./guix-vendor/rust-async-compression-0.4.21.tar.gz/tests/artifacts/lib.rs.zst
./guix-vendor/rust-async-compression-0.4.21.tar.gz/tests/artifacts/long-window-size-
./guix-vendor/rust-winapi-i686-pc-windows-gnu-0.4.0.tar.gz/lib/libwinapi_aclui.a
./guix-vendor/rust-winapi-i686-pc-windows-gnu-0.4.0.tar.gz/lib/libwinapi_activeds-
./guix-vendor/rust-winapi-i686-pc-windows-gnu-0.4.0.tar.gz/lib/libwinapi_asycfilt-
./guix-vendor/rust-winapi-i686-pc-windows-gnu-0.4.0.tar.gz/lib/libwinapi_amsi.a
...
```

Although Rust libraries are not publicly exported, we can still select them via the Guix command-line interface via a Guile expression:

```
$ guix build --expression='(@@ (gnu packages rust-crates) rust-ring-0.17.14)'
```

To unbundle most dependencies, a snippet is sufficient:

```
(define rust-curl-sys-0.4.80+curl-8.12.1
  (crate-source "curl-sys" "0.4.80+curl-8.12.1"
    "0d7ppx4kq77hc5nyff6jydmfabpgd0i3ppjvn8x0q833mhpdxsm"
    #:snippet '(delete-file-recursively "curl"))

(define rust-bzip2-sys-0.1.13+1.0.8
  (crate-source "bzip2-sys" "0.1.13+1.0.8"
    "056c39pgjh4272bds1v445f5ry64xvb0f7nph3z7860ln8rzyr2"
    #:snippet
    '(begin
      (delete-file-recursively "bzip2-1.0.8")
      (delete-file "build.rs")
      (with-output-to-file "build.rs"
        (lambda _
          (format #t "fn main() {~@
            println!(\"cargo:rustc-link-lib=bz2\");~@
            }~%"))))))))
```

In a more complex case, where unbundling one dependency requires a build process that involves other packages, we should make a full package in `(gnu packages rust-sources)` first and reference it in the imported definition.

For example, we have defined a `rust-ring-0.17` in `(gnu packages rust-sources)`, then the imported definition in `(gnu packages rust-crates)` should be modified to reference it.

```
(define rust-ring-0.17.14 rust-ring-0.17)
```

When one dependency can be safely removed, modify it to `#f`.

```
(define rust-openssl-src-300.4.2+3.4.1 #f)
```

To facilitate various tasks in the common workflow, several scripts are provided in the `etc/teams/rust` directory of Guix source tree.

2.2.1.2 Cargo Workspaces and Development Snapshots

In this example, we'll package `niri`, which depends on development snapshots (also Cargo workspaces here).

As we can't ensure compatibility of a development snapshot, before executing `cargo generate-lockfile`, we should modify `Cargo.toml` to pin it to a known working revision.

To use our packaged development snapshots, it's also necessary to modify `Cargo.toml` in a build phase, with a package-specific substitution pattern.

```
(define-public niri
  (package
    (name "niri")
    (version "25.02")
    (source (origin
      (method git-fetch)
      (uri (git-reference
        (url "https://github.com/YaLTeR/niri")
        (commit (string-append "v" version))))
      (file-name (git-file-name name version))
      (sha256
        (base32
          "0vzskaalcz6pcml687n54adjddzgf5r07gggc4fhfsa08h1wfd4r")))))
  (build-system cargo-build-system)
  (arguments
    (list #:install-source? #f
      #:phases
      #~(modify-phases %standard-phases
        (add-after 'unpack 'use-guix-vendored-dependencies
          (lambda _
            (substitute* "Cargo.toml"
              ((" # version = .*")
               "version = \"*\")
              (("git.*optional")
               "version = \"*\", optional")
              ((" ^git = .*")
               ""))))))
    (native-inputs
      (list pkg-config))
    (inputs
      (cons* clang
        libdisplay-info
        libinput-minimal
        libseat
        libxkbcommon
        mesa
```

```

    pango
    pipewire
    wayland
    (cargo-inputs 'niri)))
(home-page "https://github.com/YaLTeR/niri")
(synopsis "Scrollable-tiling Wayland compositor")
(description
  "Niri is a scrollable-tiling Wayland compositor which arranges windows in a
  scrollable format. It is considered stable for daily use and performs most
  functions expected of a Wayland compositor.")
(license license:gpl3+)))

```

niri has Cargo workspace dependencies. When packaging a Cargo workspace dependency, parameter `#:cargo-package-crates` is required.

```

(define-public rust-pipewire-0.8.0.fd3d8f7
  (let ((commit "fd3d8f7861a29c2eeaa4c393402e013578bb36d9")
        (revision "0"))
    (package
      (name "rust-pipewire")
      (version (git-version "0.8.0" revision commit))
      (source
        (origin
          (method git-fetch)
          (uri (git-reference
                (url "https://gitlab.freedesktop.org/pipewire/pipewire-rs.git")
                (commit commit))))
          (file-name (git-file-name name version))
          (sha256
            (base32 "1hzyhz7xg0mz8a5y9j6yil513p1m610q3j9pzf6q55vdh5mcn79v")))))
      (build-system cargo-build-system)
      (arguments
        (list #:skip-build? #t
              #:cargo-package-crates
              '("libspa-sys" "libspa" "pipewire-sys" "pipewire"))))
    (inputs (cargo-inputs 'rust-pipewire-0.8.0.fd3d8f7))
    (home-page "https://pipewire.org/")
    (synopsis "Rust bindings for PipeWire")
    (description "This package provides Rust bindings for PipeWire.")
    (license license:expat))))

```

Don't forget to modify all workspace members in `(gnu packages rust-crates)`:

```

(define rust-pipewire-0.8.0.fd3d8f7 package:rust-pipewire-0.8.0.fd3d8f7)
(define rust-pipewire-sys-0.8.0.fd3d8f7 package:rust-pipewire-0.8.0.fd3d8f7)
...
(define rust-libspa-0.8.0.fd3d8f7 package:rust-pipewire-0.8.0.fd3d8f7)
(define rust-libspa-sys-0.8.0.fd3d8f7 package:rust-pipewire-0.8.0.fd3d8f7)

```

2.2.1.3 Using Rust Libraries in Other Build Systems

In this example, we'll package `libchewing`, which combines two build systems.

When building Rust packages in other build systems, we need to add `rust`, and `rust:cargo` to `native-inputs`, import and use modules from both build systems, and apply necessary build phases from `cargo-build-system`.

For cross-compilation support, we'll also add `rust-sysroot` created with `(make-rust-sysroot (%current-target-system))` to `native-inputs`, and set `#:cargo-target` for `cargo-build-system`'s build phases.

```
(define-public libchewing
  (package
    (name "libchewing")
    (version "0.9.1")
    (source
      (origin
        (method git-fetch)
        (uri (git-reference
              (url "https://github.com/chewing/libchewing")
              (commit (string-append "v" version))))
        (file-name (git-file-name name version))
        (sha256
          (base32 "0gh64wvrk5pn0fhmpvj1j99d5g7f7697rk96zbkc8l72yjr819z5")))))
    (build-system cmake-build-system)
    (arguments
      (list #:imported-modules
            (append %cmake-build-system-modules
                    %cargo-build-system-modules)
            #:modules
            '(((guix build cargo-build-system) #:prefix cargo:)
              (guix build cmake-build-system)
              (guix build utils))
            #:phases
            #~(modify-phases %standard-phases
                  (add-after 'unpack 'prepare-cargo-build-system
                    (lambda args
                      (for-each
                        (lambda (phase)
                          (format #t "Running cargo phase: ~a~%" phase)
                          (apply (assoc-ref cargo:%standard-phases phase)
                                ;; For cross-compilation.
                                #:cargo-target #$(cargo-triplet)
                                args)))
                        ' (unpack-rust-crates
                          configure
                          check-for-pregenerated-files
                          patch-cargo-checksums))))))
    (native-inputs
```

```

(append
  (list rust `(.rust "cargo") )
  ;; For cross-compilation.
  (or (and=> (%current-target-system)
             (compose list make-rust-sysroot))
      '()))
(inputs
  (cons* corrosion ncurses sqlite (cargo-inputs 'libchewing)))
(synopsis "Chinese phonetic input method")
(description "Chewing is an intelligent phonetic (Zhuyin/Bopomofo) input
method, one of the most popular choices for Traditional Chinese users.")
(home-page "https://chewing.im/")
(license license:lgpl2.1+)))

```


3

Guix offers a flexible language for declaratively configuring your Guix System. This flexibility can at times be overwhelming. The purpose of this chapter is to demonstrate some advanced configuration concepts.

see Section “System Configuration” in *GNU Guix Reference Manual* for a complete reference.

3.1 TTY

While the Guix manual explains auto-login one user to *all* TTYs (see Section “auto-login to TTY” in *GNU Guix Reference Manual*), some might prefer a situation, in which one user is logged into one TTY with the other TTYs either configured to login different users or no one at all. Note that one can auto-login one user to any TTY, but it is usually advisable to avoid `tty1`, which, by default, is used to log warnings and errors.

Here is how one might set up auto login for one user to one tty:

```
(define (auto-login-to-tty config tty user)
  (if (string=? tty (mingetty-configuration-tty config))
      (mingetty-configuration
        (inherit config)
        (auto-login user))
      config))

(define %my-services
  (modify-services %base-services
    ;; ...
    (mingetty-service-type config =>
      (auto-login-to-tty
        config "tty3" "alice"))))

(operating-system
  ;; ...
  (services %my-services))
```

One could also `compose` (see Section “Higher-Order Functions” in *The Guile Reference Manual*) `auto-login-to-tty` to login multiple users to multiple ttys.

Finally, here is a note of caution. Setting up auto login to a TTY, means that anyone can turn on your computer and run commands as your regular user. However, if you have an encrypted root partition, and thus already need to enter a passphrase when the system boots, auto-login might be a convenient option.

3.2

Guix is, at its core, a source based distribution with substitutes (see Section “Substitutes” in *GNU Guix Reference Manual*), and as such building packages from their source code is an expected part of regular package installations and upgrades. Given this starting point, it makes sense that efforts are made to reduce the amount of time spent compiling packages,

and recent changes and upgrades to the building and distribution of substitutes continues to be a topic of discussion within Guix.

The kernel, while not requiring an overabundance of RAM to build, does take a rather long time on an average machine. The official kernel configuration, as is the case with many GNU/Linux distributions, errs on the side of inclusiveness, and this is really what causes the build to take such a long time when the kernel is built from source.

The Linux kernel, however, can also just be described as a regular old package, and as such can be customized just like any other package. The procedure is a little bit different, although this is primarily due to the nature of how the package definition is written.

The `linux-libre` kernel package definition is actually a procedure which creates a package.

```
(define* (make-linux-libre* version gnu-revision source supported-systems
                                #:key
                                (extra-version #f)
                                ;; A function that takes an arch and a variant.
                                ;; See kernel-config for an example.
                                (configuration-file #f)
                                (defconfig "defconfig")
                                (extra-options (default-extra-linux-options version)))
  ...)
```

The current `linux-libre` package is for the 5.15.x series, and is declared like this:

```
(define-public linux-libre-5.15
  (make-linux-libre* linux-libre-5.15-version
                     linux-libre-5.15-gnu-revision
                     linux-libre-5.15-source
                     '("x86_64-linux" "i686-linux" "armhf-linux"
                      "aarch64-linux" "riscv64-linux")
                     #:configuration-file kernel-config))
```

Any keys which are not assigned values inherit their default value from the `make-linux-libre` definition. When comparing the two snippets above, notice the code comment that refers to `#:configuration-file`. Because of this, it is not actually easy to include a custom kernel configuration from the definition, but don't worry, there are other ways to work with what we do have.

There are two ways to create a kernel with a custom kernel configuration. The first is to provide a standard `defconfig` to `customize-linux` procedure. To produce such file you need to obtain Linux source code, create a `.config` file (using `make menuconfig` command, for example) or provide one you already have. After that you will need to run `make savedefconfig` command within the kernel source tree—this will produce the `defconfig` file which we will use to build a custom kernel.

Below is a sample kernel package.

```
(define-public my-linux-libre
  (package
    (inherit (customize-linux
              #:linux linux-libre
              #:defconfig
```

```
(local-file "defconfig"))
(name "my-linux-libre"))
```

In the same directory as the file defining `my-linux-libre` is a file named `defconfig`. This way you can provide kernel options you want to have in the result kernel.

The second way to create a custom kernel is to pass a new value to the `extra-options` keyword of the `make-linux-libre` procedure. The `extra-options` keyword works with another function defined right below it:

```
(define (default-extra-linux-options version)
  `(;; https://lists.gnu.org/archive/html/guix-devel/2014-04/msg00039.html
    ("CONFIG_DEVPTS_MULTIPLE_INSTANCES" . #true)
    ;; Modules required for initrd:
    ("CONFIG_NET_9P" . m)
    ("CONFIG_NET_9P_VIRTIO" . m)
    ("CONFIG_VIRTIO_BLK" . m)
    ("CONFIG_VIRTIO_NET" . m)
    ("CONFIG_VIRTIO_PCI" . m)
    ("CONFIG_VIRTIO_BALLOON" . m)
    ("CONFIG_VIRTIO_MMIO" . m)
    ("CONFIG_FUSE_FS" . m)
    ("CONFIG_CIFS" . m)
    ("CONFIG_9P_FS" . m)))

(define (config->string options)
  (string-join (map (match-lambda
                    ((option . 'm)
                     (string-append option "=m"))
                    ((option . #true)
                     (string-append option "=y"))
                    ((option . #false)
                     (string-append option "=n"))))
               options)
               "\n"))
```

And in the custom configure script from the ‘`make-linux-libre`’ package:

```
;; Appending works even when the option wasn't in the
;; file. The last one prevails if duplicated.
(let ((port (open-file ".config" "a"))
      (extra-configuration ,(config->string extra-options)))
  (display extra-configuration port)
  (close-port port))

(invoked "make" "oldconfig")
```

So by not providing a configuration-file the `.config` starts blank, and then we write into it the collection of flags that we want. Here’s another custom kernel:

```
(define %macbook41-full-config
  (append %macbook41-config-options
```

```

%file-systems
%efi-support
%emulation
((@@ (gnu packages linux) default-extra-linux-options) version)))

(define-public linux-libre-macbook41
  ;; XXX: Access the internal 'make-linux-libre*' procedure, which is
  ;; private and unexported, and is liable to change in the future.
  ((@@ (gnu packages linux) make-linux-libre*)
   (@@ (gnu packages linux) linux-libre-version)
   (@@ (gnu packages linux) linux-libre-gnu-revision)
   (@@ (gnu packages linux) linux-libre-source)
   ("x86_64-linux")
   #:extra-version "macbook41"
   #:extra-options %macbook41-config-options))

```

In the above example `%file-systems` is a collection of flags enabling different file system support, `%efi-support` enables EFI support and `%emulation` enables a x86_64-linux machine to act in 32-bit mode also. The `default-extra-linux-options` procedure is the one defined above, which had to be used to avoid losing the default configuration options of the `extra-options` keyword.

This all sounds like it should be doable, but how does one even know which modules are required for a particular system? Two places that can be helpful in trying to answer this question is the Gentoo Handbook (<https://wiki.gentoo.org/wiki/Handbook:AMD64/Installation/Kernel>) and the documentation from the kernel itself (<https://www.kernel.org/doc/html/latest/admin-guide/README.html?highlight=localmodconfig>). From the kernel documentation, it seems that `make localmodconfig` is the command we want.

In order to actually run `make localmodconfig` we first need to get and unpack the kernel source code:

```
tar xf $(guix build linux-libre --source)
```

Once inside the directory containing the source code run `touch .config` to create an initial, empty `.config` to start with. `make localmodconfig` works by seeing what you already have in `.config` and letting you know what you're missing. If the file is blank then you're missing everything. The next step is to run:

```
guix shell -D linux-libre -- make localmodconfig
```

and note the output. Do note that the `.config` file is still empty. The output generally contains two types of warnings. The first start with "WARNING" and can actually be ignored in our case. The second read:

```
module pcspkr did not have configs CONFIG_INPUT_PCSPKR
```

For each of these lines, copy the `CONFIG_XXXX_XXXX` portion into the `.config` in the directory, and append `=m`, so in the end it looks like this:

```
CONFIG_INPUT_PCSPKR=m
CONFIG_VIRTIO=m
```

After copying all the configuration options, run `make localmodconfig` again to make sure that you don't have any output starting with "module". After all of these machine

specific modules there are a couple more left that are also needed. `CONFIG_MODULES` is necessary so that you can build and load modules separately and not have everything built into the kernel. `CONFIG_BLK_DEV_SD` is required for reading from hard drives. It is possible that there are other modules which you will need.

This post does not aim to be a guide to configuring your own kernel however, so if you do decide to build a custom kernel you'll have to seek out other guides to create a kernel which is just right for your needs.

The second way to setup the kernel configuration makes more use of Guix's features and allows you to share configuration segments between different kernels. For example, all machines using EFI to boot have a number of EFI configuration flags that they need. It is likely that all the kernels will share a list of file systems to support. By using variables it is easier to see at a glance what features are enabled and to make sure you don't have features in one kernel but missing in another.

Left undiscussed however, is Guix's `initrd` and its customization. It is likely that you'll need to modify the `initrd` on a machine using a custom kernel, since certain modules which are expected to be built may not be available for inclusion into the `initrd`.

3.3 API Guix

Historically, Guix System is centered around an `operating-system` structure. This structure contains various fields ranging from the bootloader and kernel declaration to the services to install.

Depending on the target machine, that can go from a standard `x86_64` machine to a small ARM single board computer such as the Pine64, the image constraints can vary a lot. The hardware manufacturers will impose different image formats with various partition sizes and offsets.

To create images suitable for all those machines, a new abstraction is necessary: that's the goal of the `image` record. This record contains all the required information to be transformed into a standalone image, that can be directly booted on any target machine.

```
(define-record-type* <image>
  image make-image
  image?
  (name          image-name ;symbol
    (default #f))
  (format        image-format) ;symbol
  (target        image-target
    (default #f))
  (size          image-size ;size in bytes as integer
    (default 'guess))
  (operating-system image-operating-system ;<operating-system>
    (default #f))
  (partitions    image-partitions ;list of <partition>
    (default '()))
  (compression? image-compression? ;boolean
    (default #t))
  (volatile-root? image-volatile-root? ;boolean
```

```

                                (default #t))
(substitutable?      image-substitutable? ;boolean
                                (default #t)))

```

This record contains the operating-system to instantiate. The `format` field defines the image type and can be `efi-raw`, `qcow2` or `iso9660` for instance. In the future, it could be extended to `docker` or other image types.

A new directory in the Guix sources is dedicated to images definition. For now there are four files:

- `gnu/system/images/hurd.scm`
- `gnu/system/images/pine64.scm`
- `gnu/system/images/novena.scm`
- `gnu/system/images/pinebook-pro.scm`

Let's have a look to `pine64.scm`. It contains the `pine64-barebones-os` variable which is a minimal definition of an operating-system dedicated to the **Pine A64 LTS** board.

```

(define pine64-barebones-os
  (operating-system
    (host-name "vignemale")
    (timezone "Europe/Paris")
    (locale "en_US.utf8")
    (bootloader (bootloader-configuration
                  (bootloader u-boot-pine64-lts-bootloader)
                  (targets '("/dev/vda"))))
    (initrd-modules '())
    (kernel linux-libre-arm64-generic)
    (file-systems (cons (file-system
                          (device (file-system-label "my-root"))
                          (mount-point "/")
                          (type "ext4"))
                        %base-file-systems))
    (services (cons (service agetty-service-type
                              (agetty-configuration
                                (extra-options '("-L")) ; no carrier detect
                                (baud-rate "115200")
                                (term "vt100")
                                (tty "ttyS0")))
                      %base-services)))))

```

The `kernel` and `bootloader` fields are pointing to packages dedicated to this board.

Right below, the `pine64-image-type` variable is also defined.

```

(define pine64-image-type
  (image-type
    (name 'pine64-raw)
    (constructor (cut image-with-os arm64-disk-image <>))))

```

It's using a record we haven't talked about yet, the `image-type` record, defined this way:

```

(define-record-type* <image-type>

```

```

image-type make-image-type
image-type?
(name          image-type-name) ;symbol
(constructor   image-type-constructor)) ;<operating-system> -> <image>■

```

The main purpose of this record is to associate a name to a procedure transforming an `operating-system` to an image. To understand why it is necessary, let's have a look to the command producing an image from an `operating-system` configuration file:

```
guix system image my-os.scm
```

This command expects an `operating-system` configuration but how should we indicate that we want an image targeting a Pine64 board? We need to provide an extra information, the `image-type`, by passing the `--image-type` or `-t` flag, this way:

```
guix system image --image-type=pine64-raw my-os.scm
```

This `image-type` parameter points to the `pine64-image-type` defined above. Hence, the `operating-system` declared in `my-os.scm` will be applied the `(cut image-with-os arm64-disk-image <>)` procedure to turn it into an image.

The resulting image looks like:

```

(image
  (format 'disk-image)
  (target "aarch64-linux-gnu")
  (operating-system my-os)
  (partitions
    (list (partition
            (inherit root-partition)
            (offset root-offset)))))

```

which is the aggregation of the `operating-system` defined in `my-os.scm` to the `arm64-disk-image` record.

But enough Scheme madness. What does this image API bring to the Guix user?

One can run:

```
mathieu@cervin:~$ guix system --list-image-types
```

The available image types are:

```

- unmatched-raw
- rock64-raw
- pinebook-pro-raw
- pine64-raw
- novena-raw
- hurd-raw
- hurd-qcow2
- qcow2
- iso9660
- uncompressed-iso9660
- tarball
- efi-raw
- mbr-raw

```

```
- docker
- wsl2
- raw-with-offset
- efi32-raw
```

and by writing an `operating-system` file based on `pine64-barebones-os`, you can customize your image to your preferences in a file (`my-pine-os.scm`) like this:

```
(use-modules (gnu services linux)
             (gnu system images pine64))

(let ((base-os pine64-barebones-os))
  (operating-system
    (inherit base-os)
    (timezone "America/Indiana/Indianapolis")
    (services
      (cons
        (service earlyoom-service-type
                  (earlyoom-configuration
                    (prefer-regexp "icecat|chromium"))))
        (operating-system-user-services base-os))))))
```

run:

```
guix system image --image-type=pine64-raw my-pine-os.scm
```

or,

```
guix system image --image-type=hurd-raw my-hurd-os.scm
```

to get an image that can be written directly to a hard drive and booted from.

Without changing anything to `my-hurd-os.scm`, calling:

```
guix system image --image-type=hurd-qcow2 my-hurd-os.scm
```

will instead produce a Hurd QEMU image.

3.4

The use of security keys can improve your security by providing a second authentication source that cannot be easily stolen or copied, at least for a remote adversary (something that you have), to the main secret (a passphrase – something that you know), reducing the risk of impersonation.

The example configuration detailed below showcases what minimal configuration needs to be made on your Guix System to allow the use of a Yubico security key. It is hoped the configuration can be useful for other security keys as well, with minor adjustments.

3.4.1 Configuration for use as a two-factor authenticator (2FA)

To be usable, the `udev` rules of the system should be extended with key-specific rules. The following shows how to extend your `udev` rules with the `lib/udev/rules.d/70-u2f.rules` `udev` rule file provided by the `libfido2` package from the `(gnu packages security-token)` module and add your user to the `"plugdev"` group it uses:

```
(use-package-modules ... security-token ...)
```



```

...
(operating-system
...
  (users (cons* (user-account
                  (name "your-user")
                  (group "users")
                  (supplementary-groups
                    '("wheel" "netdev" "audio" "video"
                      "plugdev"))) ;<- added system group
                  (home-directory "/home/your-user"))
            %base-user-accounts))
...
  (services
    (cons*
      ...
      (udev-rules-service 'fido2 libfido2 #:groups '("plugdev")))))

```

After re-configuring your system and re-logging in your graphical session so that the new group is in effect for your user, you can verify that your key is usable by launching:

```
guix shell ungoogled-chromium -- chromium chrome://settings/securityKeys
```

and validating that the security key can be reset via the “Reset your security key” menu. If it works, congratulations, your security key is ready to be used with applications supporting two-factor authentication (2FA).

3.4.2 Disabling OTP code generation for a Yubikey

If you use a Yubikey security key and are irritated by the spurious OTP codes it generates when inadvertently touching the key (e.g. causing you to become a spammer in the ‘#guix’ channel when discussing from your favorite IRC client!), you can disable it via the following `ykman` command:

```
guix shell python-yubikey-manager -- ykman config usb --force --disable OTP
```

Alternatively, you could use the `ykman-gui` command provided by the `yubikey-manager-qt` package and either wholly disable the ‘OTP’ application for the USB interface or, from the ‘Applications -> OTP’ view, delete the slot 1 configuration, which comes pre-configured with the Yubico OTP application.

3.4.3 Requiring a Yubikey to open a KeePassXC database

The KeePassXC password manager application has support for Yubikeys, but it requires installing a udev rules for your Guix System and some configuration of the Yubico OTP application on the key.

The necessary udev rules file comes from the `yubikey-personalization` package, and can be installed like:

```

(use-package-modules ... security-token ...)
...
(operating-system
...
  (services

```

```
(cons*
...
(udev-rules-service 'yubikey yubikey-personalization))))
```

After reconfiguring your system (and reconnecting your Yubikey), you'll then want to configure the OTP challenge/response application of your Yubikey on its slot 2, which is what KeePassXC uses. It's easy to do so via the Yubikey Manager graphical configuration tool, which can be invoked with:

```
guix shell yubikey-manager-qt -- ykman-gui
```

First, ensure 'OTP' is enabled under the 'Interfaces' tab, then navigate to 'Applications -> OTP', and click the 'Configure' button under the 'Long Touch (Slot 2)' section. Select 'Challenge-response', input or generate a secret key, and click the 'Finish' button. If you have a second Yubikey you'd like to use as a backup, you should configure it the same way, using the *same* secret key.

Your Yubikey should now be detected by KeePassXC. It can be added to a database by navigating to KeePassXC's 'Database -> Database Security...' menu, then clicking the 'Add additional protection...' button, then 'Add Challenge-Response', selecting the security key from the drop-down menu and clicking the 'OK' button to complete the setup.

3.5 DNS mcron

If your ISP (Internet Service Provider) only provides dynamic IP addresses, it can be useful to setup a dynamic DNS (Domain Name System) (also known as DDNS (Dynamic DNS)) service to associate a static host name to a public but dynamic (often changing) IP address. There are multiple existing services that can be used for this; in the following mcron job, DuckDNS (<https://duckdns.org>) is used. It should also work with other dynamic DNS services that offer a similar interface to update the IP address, such as <https://freedns.afraid.org/>, with minor adjustments.

The mcron job is provided below, where *DOMAIN* should be substituted for your own domain prefix, and the DuckDNS provided token associated to *DOMAIN* added to the `/etc/duckdns/DOMAIN.token` file.

```
(define duckdns-job
;; Update personal domain IP every 5 minutes.
#~(job '(next-minute (range 0 60 5))
#$(program-file
"duckdns-update"
(with-extensions (list guile-gnutls) ;required by (web client)
#~(begin
(use-modules (ice-9 textual-ports)
(web client))
(let ((token (string-trim-both
(call-with-input-file "/etc/duckdns/DOMAIN.token"
get-string-all)))
(query-template (string-append "https://www.duckdns.org/"
"update?domains=DOMAIN"
"&token=~a&ip="))))
(http-get (format #f query-template token))))))
```

```
"duckdns-update"
#:user "nobody"))
```

The job then needs to be added to the list of mcron jobs for your system, using something like:

```
(operating-system
 (services
  (cons* (service mcron-service-type
                 (mcron-configuration
                  (jobs (list duckdns-job ...))))
        ...
        %base-services)))
```

3.6 Wireguard VPN

To connect to a Wireguard VPN server you need the kernel module to be loaded in memory and a package providing networking tools that support it (e.g. `wireguard-tools` or `network-manager`).

Here is a configuration example for Linux-Libre < 5.6, where the module is out of tree and need to be loaded manually—following revisions of the kernel have it built-in and so don't need such configuration:

```
(use-modules (gnu))
(use-service-modules desktop)
(use-package-modules vpn)

(operating-system
 ;; ...
 (services (cons (simple-service 'wireguard-module
                                kernel-module-loader-service-type
                                '("wireguard"))
                  %desktop-services))
 (packages (cons wireguard-tools %base-packages))
 (kernel-loadable-modules (list wireguard-linux-compat)))
```

After reconfiguring and restarting your system you can either use Wireguard tools or NetworkManager to connect to a VPN server.

3.6.1 Using Wireguard tools

To test your Wireguard setup it is convenient to use `wg-quick`. Just give it a configuration file `wg-quick up ./wg0.conf`; or put that file in `/etc/wireguard` and run `wg-quick up wg0` instead.

: Be warned that the author described this command as a: “[...] very quick and dirty bash script [...]”.

3.6.2 Using NetworkManager

Thanks to NetworkManager support for Wireguard we can connect to our VPN using `nmcli` command. Up to this point this guide assumes that you're using Network Manager service

provided by `%desktop-services`. Otherwise you need to adjust your services list to load `network-manager-service-type` and reconfigure your Guix system.

To import your VPN configuration execute `nmcli import` command:

```
# nmcli connection import type wireguard file wg0.conf
Connection 'wg0' (edbee261-aa5a-42db-b032-6c7757c60fde) successfully added
```

This will create a configuration file in `/etc/NetworkManager/wg0.nmconnection`. Next connect to the Wireguard server:

```
$ nmcli connection up wg0
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/
```

By default NetworkManager will connect automatically on system boot. To change that behaviour you need to edit your config:

```
# nmcli connection modify wg0 connection.autoconnect no
```

For more specific information about NetworkManager and wireguard see this post by thaller (<https://blogs.gnome.org/thaller/2019/03/15/wireguard-in-networkmanager/>).

3.7

3.7.1 StumpWM

You could install StumpWM with a Guix system by adding `stumpwm` and optionally ``,stumpwm "lib"` packages to a system configuration file, e.g. `/etc/config.scm`.

An example configuration can look like this:

```
(use-modules (gnu))
(use-package-modules wm)

(operating-system
  ;; ...
  (packages (append (list sbcl stumpwm ``,stumpwm "lib"))
    %base-packages)))
```

By default StumpWM uses X11 fonts, which could be small or pixelated on your system. You could fix this by installing StumpWM contrib Lisp module `sbcl-ttf-fonts`, adding it to Guix system packages:

```
(use-modules (gnu))
(use-package-modules fonts wm)

(operating-system
  ;; ...
  (packages (append (list sbcl stumpwm ``,stumpwm "lib"))
    sbcl-ttf-fonts font-dejavu %base-packages)))
```

Then you need to add the following code to a StumpWM configuration file `~/.stumpwm.d/init.lisp`:

```
(require :ttf-fonts)
(setf xft:*font-dirs* '("/run/current-system/profile/share/fonts/"))
```

```
(setf clx-truetype:+font-cache-filename+ (concat (getenv "HOME")
                                                  "/.fonts/font-cache.sexp"))
(xft:cache-fonts)
(set-font (make-instance 'xft:font :family "DejaVu Sans Mono"
                               :subfamily "Book" :size 11))
```

3.7.2

Depending on your environment, locking the screen of your session might come built in or it might be something you have to set up yourself. If you use a desktop environment like GNOME or KDE, it's usually built in. If you use a plain window manager like StumpWM or EXWM, you might have to set it up yourself.

3.7.2.1 Xorg

If you use Xorg, you can use the utility `xss-lock` (<https://www.mankier.com/1/xss-lock>) to lock the screen of your session. `xss-lock` is triggered by DPMS which since Xorg 1.8 is auto-detected and enabled if ACPI is also enabled at kernel runtime.

To use `xss-lock`, you can simply execute it and put it into the background before you start your window manager from e.g. your `~/.xsession`:

```
xss-lock -- slock &
exec stumpwm
```

In this example, `xss-lock` uses `slock` to do the actual locking of the screen when it determines it's appropriate, like when you suspend your device.

For `slock` to be allowed to be a screen locker for the graphical session, it needs to be made `setuid-root` so it can authenticate users, and it needs a PAM service. This can be achieved by adding the following service to your `config.scm`:

```
(service screen-locker-service-type
  (screen-locker-configuration
    (name "slock")
    (program (file-append slock "/bin/slock"))))
```

If you manually lock your screen, e.g. by directly calling `slock` when you want to lock your screen but not suspend it, it's a good idea to notify `xss-lock` about this so no confusion occurs. This can be done by executing `xset s activate` immediately before you execute `slock`.

3.8 Guix Linode

To run Guix on a server hosted by Linode (<https://www.linode.com>), start with a recommended Debian server. We recommend using the default distro as a way to bootstrap Guix. Create your SSH keys.

```
ssh-keygen
```

Be sure to add your SSH key for easy login to the remote server. This is trivially done via Linode's graphical interface for adding SSH keys. Go to your profile and click add SSH Key. Copy into it the output of:

```
cat ~/.ssh/<username>_rsa.pub
```

Power the Linode down.

In the Linode's Storage tab, resize the Debian disk to be smaller. 30 GB free space is recommended. Then click "Add a disk", and fill out the form with the following:

- Label: "Guix"
- Filesystem: ext4
- Set it to the remaining size

In the Configurations tab, press "Edit" on the default Debian profile. Under "Block Device Assignment" click "Add a Device". It should be `/dev/sdc` and you can select the "Guix" disk. Save Changes.

Now "Add a Configuration", with the following:

- Label: Guix
- Kernel: GRUB 2 (it's at the bottom! This step is **IMPORTANT!**)
- Block device assignment:
- `/dev/sda`: Guix
- `/dev/sdb`: swap
- Root device: `/dev/sda`
- Turn off all the filesystem/boot helpers

Now power it back up, booting with the Debian configuration. Once it's running, ssh to your server via `ssh root@<your-server-IP-here>`. (You can find your server IP address in your Linode Summary section.) Now you can run the "install guix from see Section "Binary Installation" in *GNU Guix*" steps:

```
sudo apt-get install gpg
wget https://sv.gnu.org/people/viewpgp.php?user_id=15145 -qO - | gpg --import -
wget https://git.savannah.gnu.org/cgit/guix.git/plain/etc/guix-install.sh
chmod +x guix-install.sh
./guix-install.sh
guix pull
```

Now it's time to write out a config for the server. The key information is below. Save the resulting file as `guix-config.scm`.

```
(use-modules (gnu)
              (guix modules))
(use-service-modules networking
                      ssh)
(use-package-modules admin
                      package-management
                      ssh
                      tls)

(operating-system
  (host-name "my-server")
  (timezone "America/New_York")
  (locale "en_US.UTF-8")
  ;; This goofy code will generate the grub.cfg
  ;; without installing the grub bootloader on disk.
```

```

(bootloader (bootloader-configuration
  (bootloader
    (bootloader
      (inherit grub-bootloader)
      (installer #~(const #true))))))
(file-systems (cons (file-system
  (device "/dev/sda")
  (mount-point "/")
  (type "ext4"))
  %base-file-systems))

.swap-devices (list "/dev/sdb"))

(initrd-modules (cons "virtio_scsi"      ; Needed to find the disk
  %base-initrd-modules))

(users (cons (user-account
  (name "janedoe")
  (group "users")
  ;; Adding the account to the "wheel" group
  ;; makes it a sudoer.
  (supplementary-groups '("wheel"))
  (home-directory "/home/janedoe"))
  %base-user-accounts))

(packages (cons* openssh-sans-x
  %base-packages))

(services (cons*
  (service dhcp-client-service-type)
  (service openssh-service-type
    (openssh-configuration
      (openssh openssh-sans-x)
      (password-authentication? #false)
      (authorized-keys
        `(("janedoe" ,(local-file "janedoe_rsa.pub"))
          ("root" ,(local-file "janedoe_rsa.pub"))))))
  %base-services)))

```

Replace the following fields in the above configuration:

```

(host-name "my-server")      ; replace with your server name
; if you chose a linode server outside the U.S., then
; use tzselect to find a correct timezone string
(timezone "America/New_York") ; if needed replace timezone
(name "janedoe")             ; replace with your username

```

```
("janedoe" ,(local-file "janedoe_rsa.pub")) ; replace with your ssh key
("root" ,(local-file "janedoe_rsa.pub")) ; replace with your ssh key
```

The last line in the above example lets you log into the server as root and set the initial root password (see the note at the end of this recipe about root login). After you have done this, you may delete that line from your configuration and reconfigure to prevent root login.

Copy your ssh public key (eg: `~/.ssh/id_rsa.pub`) as `<your-username-here>_rsa.pub` and put `guix-config.scm` in the same directory. In a new terminal run these commands.

```
sftp root@<remote server ip address>
put /path/to/files/<username>_rsa.pub .
put /path/to/files/guix-config.scm .
```

In your first terminal, mount the guix drive:

```
mkdir /mnt/guix
mount /dev/sdc /mnt/guix
```

Due to the way we set up the bootloader section of the `guix-config.scm`, only the grub configuration file will be installed. So, we need to copy over some of the other GRUB stuff already installed on the Debian system:

```
mkdir -p /mnt/guix/boot/grub
cp -r /boot/grub/* /mnt/guix/boot/grub/
```

Now initialize the Guix installation:

```
guix system init guix-config.scm /mnt/guix
```

Ok, power it down! Now from the Linode console, select boot and select "Guix".

Once it boots, you should be able to log in via SSH! (The server config will have changed though.) You may encounter an error like:

```
$ ssh root@<server ip address>
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ECDSA key sent by the remote host is
SHA256:0B+wp33w57AnKQuHCvQP0+ZdKaqYrI/kyU7CfVbS7R4.
Please contact your system administrator.
Add correct host key in /home/joshua/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in /home/joshua/.ssh/known_hosts:3
ECDSA host key for 198.58.98.76 has changed and you have requested strict checking.
Host key verification failed.
```

Either delete `~/.ssh/known_hosts` file, or delete the offending line starting with your server IP address.

Be sure to set your password and root's password.

```
ssh root@<remote ip address>
passwd ; for the root password
passwd <username> ; for the user password
```

You may not be able to run the above commands at this point. If you have issues remotely logging into your linode box via SSH, then you may still need to set your root and

user password initially by clicking on the “Launch Console” option in your linode. Choose the “Glish” instead of “Weblish”. Now you should be able to ssh into the machine.

Hooray! At this point you can shut down the server, delete the Debian disk, and resize the Guix to the rest of the size. Congratulations!

By the way, if you save it as a disk image right at this point, you’ll have an easy time spinning up new Guix images! You may need to down-size the Guix image to 6144MB, to save it as an image. Then you can resize it again to the max size.

3.9 Guix Kimsufi

To run Guix on a server hosted by Kimsufi (<https://www.kimsufi.com/>), click on the netboot tab then select rescue64-pro and restart.

OVH will email you the credentials required to ssh into a Debian system.

Now you can run the "install guix from see Section “Binary Installation” in *GNU Guix*" steps:

```
wget https://git.savannah.gnu.org/cgit/guix.git/plain/etc/guix-install.sh
chmod +x guix-install.sh
./guix-install.sh
guix pull
```

Partition the drives and format them, first stop the raid array:

```
mdadm --stop /dev/md127
mdadm --zero-superblock /dev/sda2 /dev/sdb2
```

Then wipe the disks and set up the partitions, we will create a RAID 1 array.

```
wipefs -a /dev/sda
wipefs -a /dev/sdb
```

```
parted /dev/sda --align=opt -s -m -- mklabel gpt
parted /dev/sda --align=opt -s -m -- \
    mkpart bios_grub 1049kb 512MiB \
    set 1 bios_grub on
parted /dev/sda --align=opt -s -m -- \
    mkpart primary 512MiB -512MiB \
    set 2 raid on
parted /dev/sda --align=opt -s -m -- mkpart primary linux-swap 512MiB 100%
```

```
parted /dev/sdb --align=opt -s -m -- mklabel gpt
parted /dev/sdb --align=opt -s -m -- \
    mkpart bios_grub 1049kb 512MiB \
    set 1 bios_grub on
parted /dev/sdb --align=opt -s -m -- \
    mkpart primary 512MiB -512MiB \
    set 2 raid on
parted /dev/sdb --align=opt -s -m -- mkpart primary linux-swap 512MiB 100%
```

Create the array:

```
mdadm --create /dev/md127 --level=1 --raid-disks=2 \
```

```
--metadata=0.90 /dev/sda2 /dev/sdb2
```

Now create file systems on the relevant partitions, first the boot partitions:

```
mkfs.ext4 /dev/sda1
mkfs.ext4 /dev/sdb1
```

Then the root partition:

```
mkfs.ext4 /dev/md127
```

Initialize the swap partitions:

```
mkswap /dev/sda3
swapon /dev/sda3
mkswap /dev/sdb3
swapon /dev/sdb3
```

Mount the guix drive:

```
mkdir /mnt/guix
mount /dev/md127 /mnt/guix
```

Now is time to write an operating system declaration `os.scm` file; here is a sample:

```
(use-modules (gnu) (guix))
(use-service-modules networking ssh vpn virtualization sysctl admin mcron)
(use-package-modules ssh tls tmux vpn virtualization)
```

```
(operating-system
  (host-name "kimsufi")

  (bootloader (bootloader-configuration
    (bootloader grub-bootloader)
    (targets (list "/dev/sda" "/dev/sdb"))
    (terminal-outputs '(console))))
```

```
;; Add a kernel module for RAID-1 (aka. "mirror").
(initrd-modules (cons* "raid1" %base-initrd-modules))
```

```
(mapped-devices
  (list (mapped-device
    (source (list "/dev/sda2" "/dev/sdb2"))
    (target "/dev/md127")
    (type raid-device-mapping))))
```

```
(swap-devices
  (list (swap-space
    (target "/dev/sda3"))
    (swap-space
    (target "/dev/sdb3"))))
```

```
(issue
  ;; Default contents for /etc/issue.
  "\n")
```

```

This is the GNU system at Kimsufi.  Welcome.\n")

(file-systems (cons* (file-system
  (mount-point "/")
  (device "/dev/md127")
  (type "ext4")
  (dependencies mapped-devices))
  %base-file-systems))

(users (cons (user-account
  (name "guix")
  (comment "guix")
  (group "users")
  (supplementary-groups '("wheel")))
  (home-directory "/home/guix")))
  %base-user-accounts))

(sudoers-file
  (plain-file "sudoers" "\
root ALL=(ALL) ALL
%wheel ALL=(ALL) ALL
guix ALL=(ALL) NOPASSWD:ALL\n"))

;; Globally-installed packages.
(packages (cons* tmux gnutls wireguard-tools %base-packages))

(services
  (cons*
    (service static-networking-service-type
      (list (static-networking
        (addresses (list (network-address
          (device "enp3s0")
          (value "server-ip-address/24")))))
        (routes (list (network-route
          (destination "default")
          (gateway "server-gateway"))))
        (name-servers '("213.186.33.99")))))
    (service unattended-upgrade-service-type)
    (service openssh-service-type
      (openssh-configuration
        (openssh openssh-sans-x)
        (permit-root-login #f)
        (authorized-keys
          `(("guix" ,(plain-file "ssh-key-name.pub"
            "ssh-public-key-content")))))
      (modify-services %base-services

```

```
(sysctl-service-type
  config =>
  (sysctl-configuration
    (settings (append '(("net.ipv6.conf.all.autoconf" . "0")
      ("net.ipv6.conf.all.accept_ra" . "0"))
      %default-sysctl-settings))))))
```

Don't forget to substitute the *server-ip-address*, *server-gateway*, *ssh-key-name* and *ssh-public-key-content* variables with your own values.

The gateway is the last usable IP in your block so if you have a server with an IP of '37.187.79.10' then its gateway will be '37.187.79.254'.

Transfer your operating system declaration *os.scm* file on the server via the *scp* or *sftp* commands.

Now all that is left is to install Guix with a *guix system init* and restart.

However we first need to set up a chroot, because the root partition of the rescue system is mounted on an aufs partition and if you try to install Guix it will fail at the GRUB install step complaining about the canonical path of "aufs".

Install packages that will be used in the chroot:

```
guix install bash-static parted util-linux-with-udev coreutils guix
```

Then run the following to create directories needed for the chroot:

```
cd /mnt && \
mkdir -p bin etc gnu/store root/.guix-profile/ root/.config/guix/current \
var/guix proc sys dev
```

Copy the host *resolv.conf* in the chroot:

```
cp /etc/resolv.conf etc/
```

Mount block devices, the store and its database and the current guix config:

```
mount --rbind /proc /mnt/proc
mount --rbind /sys /mnt/sys
mount --rbind /dev /mnt/dev
mount --rbind /var/guix/ var/guix/
mount --rbind /gnu/store gnu/store/
mount --rbind /root/.config/ root/.config/
mount --rbind /root/.guix-profile/bin/ bin
mount --rbind /root/.guix-profile root/.guix-profile/
```

Chroot in */mnt* and install the system:

```
chroot /mnt/ /bin/bash
```

```
guix system init /root/os.scm /guix
```

Finally, from the web user interface (UI), change 'netboot' to 'boot to disk' and restart (also from the web UI).

Wait a few minutes and try to ssh with *ssh guix@server-ip-address> -i path-to-your-ssh-key*

You should have a Guix system up and running on Kimsufi; congratulations!

3.10

To bind mount a file system, one must first set up some definitions before the `operating-system` section of the system definition. In this example we will bind mount a folder from a spinning disk drive to `/tmp`, to save wear and tear on the primary SSD, without dedicating an entire partition to be mounted as `/tmp`.

First, the source drive that hosts the folder we wish to bind mount should be defined, so that the bind mount can depend on it.

```
(define source-drive ;; "source-drive" can be named anything you want.
  (file-system
    (device (uuid "UUID goes here"))
    (mount-point "/path-to-spinning-disk-goes-here")
    (type "ext4"))) ;Make sure to set this to the appropriate type for your drive.■
```

The source folder must also be defined, so that guix will know it's not a regular block device, but a folder.

```
;; "source-directory" can be named any valid variable name.
(define (%source-directory) "/path-to-spinning-disk-goes-here/tmp")
```

Finally, inside the `file-systems` definition, we must add the mount itself.

```
(file-systems (cons*

  ...<other drives omitted for clarity>...

  ;; Must match the name you gave the source drive in the earlier definition
  source-drive

  (file-system
    ;; Make sure "source-directory" matches your earlier definition.■
    (device (%source-directory))
    (mount-point "/tmp")
    ;; We are mounting a folder, not a partition, so this type needs to be "none"
    (type "none")
    (flags '(bind-mount))
    ;; Ensure "source-drive" matches what you've named the variable for the source
    (dependencies (list source-drive))
  )

  ...<other drives omitted for clarity>...

))
```

3.11 Tor

Guix daemon can use a HTTP proxy to get substitutes, here we are configuring it to get them via Tor.

: *Not all* Guix daemon's traffic will go through Tor! Only HTTP/HTTPS will get proxied; FTP, Git protocol, SSH, etc connections will still go through the

clearnet. Again, this configuration isn't foolproof some of your traffic won't get routed by Tor at all. Use it at your own risk.

Also note that the procedure described here applies only to package substitution. When you update your guix distribution with `guix pull`, you still need to use `torsocks` if you want to route the connection to guix's git repository servers through Tor.

Guix's substitute server is available as a Onion service, if you want to use it to get your substitutes through Tor configure your system as follow:

```
(use-modules (gnu))
(use-service-module base networking)

(operating-system
  ...
  (services
    (cons
      (service tor-service-type
        (tor-configuration
          (config-file (plain-file "tor-config"
                                "HTTPTunnelPort 127.0.0.1:9250"))))
      (modify-services %base-services
        (guix-service-type
          config => (guix-configuration
                     (inherit config)
                     ;; ci.guix.gnu.org's Onion service
                     (substitute-urls "\
https://4zwzi66wwdaalbhgnix55ea3ab4pvvw66112ow53kjub6se4q2bclcyd.onion")
                     (http-proxy "http://localhost:9250"))))))))
```

This will keep a tor process running that provides a HTTP CONNECT tunnel which will be used by `guix-daemon`. The daemon can use other protocols than HTTP(S) to get remote resources, request using those protocols won't go through Tor since we are only setting a HTTP tunnel here. Note that `substitutes-urls` is using HTTPS and not HTTP or it won't work, that's a limitation of Tor's tunnel; you may want to use `privoxy` instead to avoid such limitations.

If you don't want to always get substitutes through Tor but using it just some of the times, then skip the `guix-configuration`. When you want to get a substitute from the Tor tunnel run:

```
sudo herd set-http-proxy guix-daemon http://localhost:9250
guix build \
  --substitute-urls=https://4zwzi66wwdaalbhgnix55ea3ab4pvvw66112ow53kjub6se4q2bclcyd.o
```

3.12 NGINX Lua

NGINX could be extended with Lua scripts.

Guix provides NGINX service with ability to load Lua module and specific Lua packages, and reply to requests by evaluating Lua scripts.

The following example demonstrates system definition with configuration to evaluate `index.lua` Lua script on HTTP request to `http://localhost/hello` endpoint:

```
local shell = require "resty.shell"

local stdin = ""
local timeout = 1000 -- ms
local max_size = 4096 -- byte

local ok, stdout, stderr, reason, status =
    shell.run([[/run/current-system/profile/bin/ls /tmp]], stdin, timeout, max_size)

ngx.say(stdout)

(use-modules (gnu))
(use-service-modules #;... web)
(use-package-modules #;... lua)
(operating-system
  ;; ...
  (services
    ;; ...
    (service nginx-service-type
      (nginx-configuration
        (modules
          (list
            (file-append nginx-lua-module "/etc/nginx/modules/ngx_http_lua_module.so")
            (lua-package-path (list lua-resty-core
                                  lua-resty-lrucache
                                  lua-resty-signal
                                  lua-tablepool
                                  lua-resty-shell)))
          (lua-package-cpath (list lua-resty-signal)))
        (server-blocks
          (list (nginx-server-configuration
                (server-name '("localhost"))
                (listen '("80"))
                (root "/etc")
                (locations (list
                  (nginx-location-configuration
                    (uri "/hello")
                    (body (list #~(format #f "content_by_lua_file ~s;"
                                         #$(local-file "index.lua"))))))
```

3.13 Bluetooth-

MPD, the Music Player Daemon, is a flexible server-side application for playing music. Client programs on different machines on the network — a mobile phone, a laptop, a desktop workstation — can connect to it to control the playback of audio files from your

local music collection. MPD decodes the audio files and plays them back on one or many outputs.

By default MPD will play to the default audio device. In the example below we make things a little more interesting by setting up a headless music server. There will be no graphical user interface, no Pulseaudio daemon, and no local audio output. Instead we will configure MPD with two outputs: a bluetooth speaker and a web server to serve audio streams to any streaming media player.

Bluetooth is often rather frustrating to set up. You will have to pair your Bluetooth device and make sure that the device is automatically connected as soon as it powers on. The Bluetooth system service returned by the `bluetooth-service` procedure provides the infrastructure needed to set this up.

Reconfigure your system with at least the following services and packages:

```
(operating-system
;; ...
(packages (cons* bluez bluez-alsa
                  %base-packages))

(services
;; ...
(dbus-service #:services (list bluez-alsa))
(bluetooth-service #:auto-enable? #t)))
```

Start the `bluetooth` service and then use `bluetoothctl` to scan for Bluetooth devices. Try to identify your Bluetooth speaker and pick out its device ID from the resulting list of devices that is indubitably dominated by a baffling smorgasbord of your neighbors' home automation gizmos. This only needs to be done once:

```
$ bluetoothctl
[NEW] Controller 00:11:22:33:95:7F BlueZ 5.40 [default]

[bluetooth]# power on
[bluetooth]# Changing power on succeeded

[bluetooth]# agent on
[bluetooth]# Agent registered

[bluetooth]# default-agent
[bluetooth]# Default agent request successful

[bluetooth]# scan on
[bluetooth]# Discovery started
[CHG] Controller 00:11:22:33:95:7F Discovering: yes
[NEW] Device AA:BB:CC:A4:AA:CD My Bluetooth Speaker
[NEW] Device 44:44:FF:2A:20:DC My Neighbor's TV
...

[bluetooth]# pair AA:BB:CC:A4:AA:CD
Attempting to pair with AA:BB:CC:A4:AA:CD
```



```
[CHG] Device AA:BB:CC:A4:AA:CD Connected: yes
```

```
[My Bluetooth Speaker]# [CHG] Device AA:BB:CC:A4:AA:CD UUIDs: 0000110b-0000-1000-8000-
```

```
[CHG] Device AA:BB:CC:A4:AA:CD UUIDs: 0000110c-0000-1000-8000-00xxxxxxxxx█
```

```
[CHG] Device AA:BB:CC:A4:AA:CD UUIDs: 0000110e-0000-1000-8000-00xxxxxxxxx█
```

```
[CHG] Device AA:BB:CC:A4:AA:CD Paired: yes
```

```
Pairing successful
```

```
[CHG] Device AA:BB:CC:A4:AA:CD Connected: no
```

```
[bluetooth]#
```

```
[bluetooth]# trust AA:BB:CC:A4:AA:CD
```

```
[bluetooth]# [CHG] Device AA:BB:CC:A4:AA:CD Trusted: yes
```

```
Changing AA:BB:CC:A4:AA:CD trust succeeded
```

```
[bluetooth]#
```

```
[bluetooth]# connect AA:BB:CC:A4:AA:CD
```

```
Attempting to connect to AA:BB:CC:A4:AA:CD
```

```
[bluetooth]# [CHG] Device AA:BB:CC:A4:AA:CD RSSI: -63
```

```
[CHG] Device AA:BB:CC:A4:AA:CD Connected: yes
```

```
Connection successful
```

```
[My Bluetooth Speaker]# scan off
```

```
[CHG] Device AA:BB:CC:A4:AA:CD RSSI is nil
```

```
Discovery stopped
```

```
[CHG] Controller 00:11:22:33:95:7F Discovering: no
```

Congratulations, you can now automatically connect to your Bluetooth speaker!

It is now time to configure ALSA to use the *bluealsa* Bluetooth module, so that you can define an ALSA pcm device corresponding to your Bluetooth speaker. For a headless server using *bluealsa* with a fixed Bluetooth device is likely simpler than configuring Pulseaudio and its stream switching behavior. We configure ALSA by crafting a custom *alsa-configuration* for the *alsa-service-type*. The configuration will declare a *pcm* type *bluealsa* from the *bluealsa* module provided by the *bluez-alsa* package, and then define a *pcm* device of that type for your Bluetooth speaker.

All that is left then is to make MPD send audio data to this ALSA device. We also add a secondary MPD output that makes the currently played audio files available as a stream through a web server on port 8080. When enabled a device on the network could listen to the audio stream by connecting any capable media player to the HTTP server on port 8080, independent of the status of the Bluetooth speaker.

What follows is the outline of an *operating-system* declaration that should accomplish the above-mentioned tasks:

```
(use-modules (gnu))
(use-service-modules audio dbus sound #;... etc)
(use-package-modules audio linux #;... etc)
(operating-system
  ;; ...
```

```

/packages (cons* bluez bluez-alsa
            %base-packages))

(services
;; ...
(service mpd-service-type
  (mpd-configuration
    (user "your-username")
    (music-dir "/path/to/your/music")
    (address "192.168.178.20")
    (outputs (list (mpd-output
      (type "alsa")
      (name "MPD")
      (extra-options
        ;; Use the same name as in the ALSA
        ;; configuration below.
        '((device . "pcm.btspeaker")))))
      (mpd-output
        (type "httpd")
        (name "streaming")
        (enabled? #false)
        (always-on? #true)
        (tags? #true)
        (mixer-type 'null)
        (extra-options
          '((encoder . "vorbis")
            (port . "8080")
            (bind-to-address . "192.168.178.20")
            (max-clients . "0") ;no limit
            (quality . "5.0")
            (format . "44100:16:1"))))))))

(dbus-service #:services (list bluez-alsa))
(bluetooth-service #:auto-enable? #t)
(service alsa-service-type
  (alsa-configuration
    (pulseaudio? #false) ;we don't need it
    (extra-options
      #~(string-append "\
# Declare Bluetooth audio device type \"bluealsa\" from bluealsa module
pcm_type.bluealsa {
  lib \"\"
  #$(file-append bluez-alsa "/lib/alsa-lib/libasound_module_pcm_bluealsa.so") "\
}

# Declare control device type \"bluealsa\" from the same module
ctl_type.bluealsa {
  lib \"\"
  #$(file-append bluez-alsa "/lib/alsa-lib/libasound_module_ctl_bluealsa.so") "\

```

```
}

# Define the actual Bluetooth audio device.
pcm.btspeaker {
    type bluealsa
    device \"AA:BB:CC:A4:AA:CD\" # unique device identifier
    profile \"a2dp\"
}

# Define an associated controller.
ctl.btspeaker {
    type bluealsa
}
"))))))
```

Enjoy the music with the MPD client of your choice or a media player capable of streaming via HTTP!

4

The kernel Linux provides a number of shared facilities that are available to processes in the system. These facilities include a shared view on the file system, other processes, network devices, user and group identities, and a few others. Since Linux 3.19 a user can choose to *unshare* some of these shared facilities for selected processes, providing them (and their child processes) with a different view on the system.

A process with an unshared **mount** namespace, for example, has its own view on the file system — it will only be able to see directories that have been explicitly bound in its mount namespace. A process with its own **proc** namespace will consider itself to be the only process running on the system, running as PID 1.

Guix uses these kernel features to provide fully isolated environments and even complete Guix System containers, lightweight virtual machines that share the host system’s kernel. This feature comes in especially handy when using Guix on a foreign distribution to prevent interference from foreign libraries or configuration files that are available system-wide.

4.1 Guix

The easiest way to get started is to use **guix shell** with the **--container** option. See Section “Invoking guix shell” in *GNU Guix Reference Manual* for a reference of valid options.

The following snippet spawns a minimal shell process with most namespaces unshared from the system. The current working directory is visible to the process, but anything else on the file system is unavailable. This extreme isolation can be very useful when you want to rule out any sort of interference from environment variables, globally installed libraries, or configuration files.

```
guix shell --container
```

It is a bleak environment, barren, desolate. You will find that not even the GNU coreutils are available here, so to explore this deserted wasteland you need to use built-in shell commands. Even the usually gigantic **/gnu/store** directory is reduced to a faint shadow of itself.

```
$ echo /gnu/store/*
/gnu/store/...-gcc-10.3.0-lib
/gnu/store/...-glibc-2.33
/gnu/store/...-bash-static-5.1.8
/gnu/store/...-ncurses-6.2.20210619
/gnu/store/...-bash-5.1.8
/gnu/store/...-profile
/gnu/store/...-readline-8.1.1
```

There isn’t much you can do in an environment like this other than exiting it. You can use **^D** or **exit** to terminate this limited shell environment.

You can make other directories available inside of the container environment; use **--expose=DIRECTORY** to bind-mount the given directory as a read-only location inside the container, or use **--share=DIRECTORY** to make the location writable. With an additional mapping argument after the directory name you can control the name of the directory

inside the container. In the following example we map `/etc` on the host system to `/the/host/etc` inside a container in which the GNU coreutils are installed.

```
$ guix shell --container --share=/etc=/the/host/etc coreutils
$ ls /the/host/etc
```

Similarly, you can prevent the current working directory from being mapped into the container with the `--no-cwd` option. Another good idea is to create a dedicated directory that will serve as the container's home directory, and spawn the container shell from that directory.

On a foreign system a container environment can be used to compile software that cannot possibly be linked with system libraries or with the system's compiler toolchain. A common use-case in a research context is to install packages from within an R session. Outside of a container environment there is a good chance that the foreign compiler toolchain and incompatible system libraries are found first, resulting in incompatible binaries that cannot be used by R. In a container shell this problem disappears, as system libraries and executables simply aren't available due to the unshared `mount` namespace.

Let's take a comprehensive manifest providing a comfortable development environment for use with R:

```
(specifications->manifest
  (list "r-minimal"

        ;; base packages
        "bash-minimal"
        "glibc-locales"
        "nss-certs"

        ;; Common command line tools lest the container is too empty.
        "coreutils"
        "grep"
        "which"
        "wget"
        "sed"

        ;; R markdown tools
        "pandoc"

        ;; Toolchain and common libraries for "install.packages"
        "gcc-toolchain@10"
        "gfortran-toolchain"
        "gawk"
        "tar"
        "gzip"
        "unzip"
        "make"
        "cmake"
        "pkg-config"
        "cairo"
```

```
"libxt"
"openssl"
"curl"
"zlib"))
```

Let's use this to run R inside a container environment. For convenience we share the `net` namespace to use the host system's network interfaces. Now we can build R packages from source the traditional way without having to worry about ABI mismatch or incompatibilities.

```
$ guix shell --container --network --manifest=manifest.scm -- R

R version 4.2.1 (2022-06-23) -- "Funny-Looking Kid"
Copyright (C) 2022 The R Foundation for Statistical Computing
...
> e <- Sys.getenv("GUIX_ENVIRONMENT")
> Sys.setenv(GIT_SSL_CAINFO=paste0(e, "/etc/ssl/certs/ca-certificates.crt"))
> Sys.setenv(SSL_CERT_FILE=paste0(e, "/etc/ssl/certs/ca-certificates.crt"))
> Sys.setenv(SSL_CERT_DIR=paste0(e, "/etc/ssl/certs"))
> install.packages("Cairo", lib=paste0(getwd()))
...
* installing *source* package 'Cairo' ...
...
* DONE (Cairo)

The downloaded source packages are in
'/tmp/RtmpCuwdwM/downloaded_packages'
> library("Cairo", lib=getwd())
> # success!
```

Using container shells is fun, but they can become a little cumbersome when you want to go beyond just a single interactive process. Some tasks become a lot easier when they sit on the rock solid foundation of a proper Guix System and its rich set of system services. The next section shows you how to launch a complete Guix System inside of a container.

4.2 Guix System

The Guix System provides a wide array of interconnected system services that are configured declaratively to form a dependable stateless GNU System foundation for whatever tasks you throw at it. Even when using Guix on a foreign distribution you can benefit from the design of Guix System by running a system instance as a container. Using the same kernel features of unshared namespaces mentioned in the previous section, the resulting Guix System instance is isolated from the host system and only shares file system locations that you explicitly declare.

A Guix System container differs from the shell process created by `guix shell --container` in a number of important ways. While in a container shell the containerized process is a Bash shell process, a Guix System container runs the Shepherd as PID 1. In a system container all system services (see Section “Services” in *GNU Guix Reference Manual*) are set up just as they would be on a Guix System in a virtual machine or on

bare metal—this includes daemons managed by the GNU Shepherd (see Section “Shepherd Services” in *GNU Guix Reference Manual*) as well as other kinds of extensions to the operating system (see Section “Service Composition” in *GNU Guix Reference Manual*).

The perceived increase in complexity of running a Guix System container is easily justified when dealing with more complex applications that have higher or just more rigid requirements on their execution contexts—configuration files, dedicated user accounts, directories for caches or log files, etc. In Guix System the demands of this kind of software are satisfied through the deployment of system services.

4.2.1

A good example might be a PostgreSQL database server. Much of the complexity of setting up such a database server is encapsulated in this deceptively short service declaration:

```
(service postgresql-service-type
  (postgresql-configuration
    (postgresql postgresql-10)))
```

A complete operating system declaration for use with a Guix System container would look something like this:

```
(use-modules (gnu))
(use-package-modules databases)
(use-service-modules databases)

(operating-system
  (host-name "container")
  (timezone "Europe/Berlin")
  (file-systems (cons (file-system
    (device (file-system-label "does-not-matter"))
    (mount-point "/")
    (type "ext4"))
    %base-file-systems))
  (bootloader (bootloader-configuration
    (bootloader grub-bootloader)
    (targets '("/dev/sdX"))))
  (services
    (cons* (service postgresql-service-type
      (postgresql-configuration
        (postgresql postgresql-14)
        (config-file
          (postgresql-config-file
            (log-destination "stderr")
            (hba-file
              (plain-file "pg_hba.conf"
                "\
local all all trust
host all all 10.0.0.1/32  trust"))
        (extra-config
          '(("listen_addresses" "*"))
```

```

        ("log_directory"      "/var/log/postgresql"))))))))■
(service postgresql-role-service-type
  (postgresql-role-configuration
    (roles
      (list (postgresql-role
              (name "test")
              (create-database? #t)))))
    %base-services)))

```

With `postgresql-role-service-type` we define a role “test” and create a matching database, so that we can test right away without any further manual setup. The `postgresql-config-file` settings allow a client from IP address 10.0.0.1 to connect without requiring authentication—a bad idea in production systems, but convenient for this example.

Let’s build a script that will launch an instance of this Guix System as a container. Write the `operating-system` declaration above to a file `os.scm` and then use `guix system container` to build the launcher. (see Section “Invoking guix system” in *GNU Guix Reference Manual*).

```

$ guix system container os.scm
The following derivations will be built:
  /gnu/store/...-run-container.drv
  ...
building /gnu/store/...-run-container.drv...
/gnu/store/...-run-container

```

Now that we have a launcher script we can run it to spawn the new system with a running PostgreSQL service. Note that due to some as yet unresolved limitations we need to run the launcher as the root user, for example with `sudo`.

```

$ sudo /gnu/store/...-run-container
system container is running as PID 5983
...

```

Background the process with `Ctrl-z` followed by `bg`. Note the process ID in the output; we will need it to connect to the container later. You know what? Let’s try attaching to the container right now. We will use `nsenter`, a tool provided by the `util-linux` package:

```

$ guix shell util-linux
$ sudo nsenter -a -t 5983
root@container /# pgrep -a postgres
49 /gnu/store/...-postgresql-14.4/bin/postgres -D /var/lib/postgresql/data --config-fi
51 postgres: checkpointer
52 postgres: background writer
53 postgres: walwriter
54 postgres: autovacuum launcher
55 postgres: stats collector
56 postgres: logical replication launcher
root@container /# exit

```

The PostgreSQL service is running in the container!

4.2.2

What good is a Guix System running a PostgreSQL database service as a container when we can only talk to it with processes originating in the container? It would be much better if we could talk to the database over the network.

The easiest way to do this is to create a pair of connected virtual Ethernet devices (known as `veth`). We move one of the devices (`ceth-test`) into the `net` namespace of the container and leave the other end (`veth-test`) of the connection on the host system.

```
pid=5983
ns="guix-test"
host="veth-test"
client="ceth-test"

# Attach the new net namespace "guix-test" to the container PID.
sudo ip netns attach $ns $pid

# Create the pair of devices
sudo ip link add $host type veth peer name $client

# Move the client device into the container's net namespace
sudo ip link set $client netns $ns
```

Then we configure the host side:

```
sudo ip link set $host up
sudo ip addr add 10.0.0.1/24 dev $host
```

...and then we configure the client side:

```
sudo ip netns exec $ns ip link set lo up
sudo ip netns exec $ns ip link set $client up
sudo ip netns exec $ns ip addr add 10.0.0.2/24 dev $client
```

At this point the host can reach the container at IP address 10.0.0.2, and the container can reach the host at IP 10.0.0.1. This is all we need to talk to the database server inside the container from the host system on the outside.

```
$ psql -h 10.0.0.2 -U test
psql (14.4)
Type "help" for help.
```

```
test=> CREATE TABLE hello (who TEXT NOT NULL);
CREATE TABLE
test=> INSERT INTO hello (who) VALUES ('world');
INSERT 0 1
test=> SELECT * FROM hello;
   who
-----
 world
(1 row)
```

Now that we're done with this little demonstration let's clean up:

```
sudo kill $pid
```

```
sudo ip netns del $ns  
sudo ip link del $host
```

5

Guix can produce disk images (see Section “Invoking guix system” in *GNU Guix Reference Manual*) that can be used with virtual machines solutions such as virt-manager, GNOME Boxes or the more bare QEMU, among others.

This chapter aims to provide hands-on, practical examples that relates to the usage and configuration of virtual machines on a Guix System.

5.1 QEMU

By default, QEMU uses a so-called “user mode” host network back-end, which is convenient as it does not require any configuration. Unfortunately, it is also quite limited. In this mode, the guest VM (virtual machine) can access the network the same way the host would, but it cannot be reached from the host. Additionally, since the QEMU user networking mode relies on ICMP, ICMP-based networking tools such as `ping` do *not* work in this mode. Thus, it is often desirable to configure a network bridge, which enables the guest to fully participate in the network. This is necessary, for example, when the guest is to be used as a server.

5.1.1 Creating a network bridge interface

There are many ways to create a network bridge. The following command shows how to use NetworkManager and its `nmcli` command line interface (CLI) tool, which should already be available if your operating system declaration is based on one of the desktop templates:

```
# nmcli con add type bridge con-name br0 ifname br0
```

To have this bridge be part of your network, you must associate your network bridge with the Ethernet interface used to connect with the network. Assuming your interface is named `enp2s0`, the following command can be used to do so:

```
# nmcli con add type bridge-slave ifname enp2s0 master br0
```

: Only Ethernet interfaces can be added to a bridge. For wireless interfaces, consider the routed network approach detailed in See Section 5.2 [`libvirt`], page 63.

By default, the network bridge will allow your guests to obtain their IP address via DHCP, if available on your local network. For simplicity, this is what we will use here. To easily find the guests, they can be configured to advertise their host names via mDNS.

5.1.2 Configuring the QEMU bridge helper script

QEMU comes with a helper program to conveniently make use of a network bridge interface as an unprivileged user see Section “Network options” in *QEMU Documentation*. The binary must be made `setuid root` for proper operation; this can be achieved by adding it to the `privileged-programs` field of your (host) `operating-system` definition, as shown below:

```
(privileged-programs
 (cons (privileged-program
        (program (file-append qemu "/libexec/qemu-bridge-helper"))
        (setuid? #t)))
```

```
%default-privileged-programs))
```

The file `/etc/qemu/bridge.conf` must also be made to allow the bridge interface, as the default is to deny all. Add the following to your list of services to do so:

```
(extra-special-file "/etc/qemu/host.conf"
  (plain-file "host.conf" "allow br0\n"))
```

5.1.3 Invoking QEMU with the right command line options

When invoking QEMU, the following options should be provided so that the network bridge is used, after having selected a unique MAC address for the guest.

: By default, a single MAC address is used for all guests, unless provided. Failing to provide different MAC addresses to each virtual machine making use of the bridge would cause networking issues.

```
$ qemu-system-x86_64 [...] \
  -device virtio-net-pci,netdev=user0,mac=XX:XX:XX:XX:XX:XX \
  -netdev bridge,id=user0,br=br0 \
  [...]
```

To generate MAC addresses that have the QEMU registered prefix, the following snippet can be employed:

```
mac_address="52:54:00:$(dd if=/dev/urandom bs=512 count=1 2>/dev/null \
  | md5sum \
  | sed -E 's/^(..)(..)(..).*$/\1:\2:\3/')"

echo $mac_address
```

5.1.4 Networking issues caused by Docker

If you use Docker on your machine, you may experience connectivity issues when attempting to use a network bridge, which are caused by Docker also relying on network bridges and configuring its own routing rules. The solution is add the following `iptables` snippet to your `operating-system` declaration:

```
(service iptables-service-type
  (iptables-configuration
    (ipv4-rules (plain-file "iptables.rules" "\
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
-A FORWARD -i br0 -o br0 -j ACCEPT
COMMIT
"))
```

5.2 libvirt

If the machine hosting your virtual machines is connected wirelessly to the network, you won't be able to use a true network bridge as explained in the preceding section (see Section 5.1 [QEMU], page 62). In this case, the next best option is to use a *virtual network switch* with static routing and to configure a libvirt-powered virtual machine to use it (via

the `virt-manager` GUI for example). This is similar to the default mode of operation of QEMU/libvirt, except that instead of using NAT (Network Address Translation), it relies on static routes to join the VM (virtual machine) IP address to the LAN (local area network). This provides two-way connectivity to and from the virtual machine, which is needed for exposing services hosted on the virtual machine.

5.2.1 Creating a virtual network switch

A virtual network switch consists of a virtual network device called a ‘virtual bridge’, DHCP server (dnsmasq) and firewall rules (iptables). See the libvirt Wiki article on Virtual Networking (<https://wiki.libvirt.org/VirtualNetworking.html>) for more details on the modes of operation, management and implementation of virtual network switches.

The `virsh` command, provided by the `libvirt` package, makes it very easy to create a virtual switch. You first need to choose a network subnet for your virtual switch; if your home LAN is in the ‘192.168.1.0/24’ network, you could opt to use e.g. ‘192.168.2.0/24’. Define an XML file, e.g. `/tmp/virbr0.xml`, containing the following:

```
<network>
  <name>virbr0</name>
  <bridge name="virbr0" />
  <forward mode="route"/>
  <ip address="192.168.2.0" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.2.1" end="192.168.2.254"/>
    </dhcp>
  </ip>
</network>
```

Then create and configure the interface using the `virsh` command, as root:

```
virsh net-define /tmp/virbr0.xml
virsh net-autostart virbr0
virsh net-start virbr0
```

The ‘`virbr0`’ interface should now be visible e.g. via the ‘`ip address`’ command. It will be automatically started every time your libvirt virtual machine is started.

5.2.2 Configuring the static routes for your virtual bridge

If you configured your virtual machine to use your newly created ‘`virbr0`’ virtual bridge interface, it should already receive an IP via DHCP such as ‘192.168.2.15’ and be reachable from the server hosting it, e.g. via ‘`ping 192.168.2.15`’. There’s one last configuration needed so that the VM can reach the external network: adding static routes to the network’s router.

In this example, the LAN network is ‘192.168.1.0/24’ and the router configuration web page may be accessible via e.g. the <http://192.168.1.1> page. On a router running the libreCMC (<https://librecmc.org/>) firmware, you would navigate to the Network → Static Routes page (<https://192.168.1.1/cgi-bin/luci/admin/network/routes>), and you would add a new entry to the ‘Static IPv4 Routes’ with the following information:

```
‘      lan
‘      192.168.2.0
```

```

‘ IPv4’      255.255.255.0
‘ IPv4’      server-ip
‘Route type’
            unicast

```

where *server-ip* is the IP address of the machine hosting the VMs, which should be static.

After saving/applying this new static route, external connectivity should work from within your VM; you can e.g. run ‘`ping gnu.org`’ to verify that it functions correctly.

5.3 NAT libvirt

As mentioned in the preceding section (see Section 5.2 [libvirt], page 63), libvirt allows virtual networks to be defined via XML files and managed by the `virsh` command. The details of the creation and removal of virtual network switches are handled by libvirt, so the user does not have to deal with them.

However, libvirt’s handling of virtual network switches can sometimes clash with more complex networking setups. In particular, the iptables rules inserted by libvirt for switches operating in the NAT mode can clash with existing iptables/nftables rules, leading to insecure or broken packet filtering.

In such cases, the only solution is to manually set up a virtual network switch. This section will provide instructions on how to do so using Guix System services.

5.3.1 Creating the virtual network bridge

The `static-networking-service-type` can be used to create a virtual network bridge and assign an IP address to it:

```

(service static-networking-service-type
  (list (static-networking
        ;; The default provision is 'networking; if you're using any
        ;; other service with this provision, such as
        ;; `network-manager-service-type`, then you need to change the
        ;; default.
        (provision '(static-networking))
        (links
          (list (network-link
                 (name "virbr0")
                 (type 'bridge)
                 (arguments '())))))
        (addresses
          (list (network-address
                 (device "virbr0")
                 (value "192.168.10.1/24"))))))))

```

5.3.2 Running dnsmasq for the virtual network bridge

The `dnsmasq-service-type` can be used to provide DNS and DHCP for guests connected to this virtual network switch:

```

(service dnsmasq-service-type

```

```
(dnsmasq-configuration
;; You can have multiple instances of `dnsmasq-service-type` as long
;; as each one has a different shepherd-provision.
(shepherd-provision '(dnsmasq-virbr0))
(extra-options (list
                ;; Only bind to the virtual bridge. This
                ;; avoids conflicts with other running
                ;; dnsmasq instances.
                "--except-interface=lo"
                "--interface=virbr0"
                "--bind-dynamic"
                ;; IPv4 addresses to offer to VMs. This
                ;; should match the chosen subnet.
                "--dhcp-range=192.168.10.2,192.168.10.254"))))
```

5.3.3 Configuring NAT for the virtual network switch

If you intend to use the virtual network switch in NAT mode, you will need to use `nftables` (or `iptables`) rules to set up IP masquerading. The following example shows how to use `nftables-service-type` to do this:

```
(service nftables-service-type
  (nftables-configuration
    (ruleset
      (plain-file "nftables.conf"
        "\n
table inet filter {

  chain input {
    type filter hook input priority filter; policy drop;
    # Add your existing packet filtering rules here...
    iifname virbr0 udp dport 67 counter accept comment \"allow dhcp on virbr0\"
    iifname virbr0 meta l4proto {tcp, udp} th dport 53 accept \\
      comment \"allow dns on virbr0\"
  }

  chain forward {
    type filter hook forward priority filter; policy drop;
    # Add your existing forwarding rules here...
    iifname virbr0 accept comment \"allow outbound traffic from virbr0\"
    oifname virbr0 ct state {established, related } accept \\
      comment \"allow established traffic to virbr0\"
  }
}

table inet nat {
  chain postrouting {
```

```

type nat hook postrouting priority srcnat; policy accept;
# Add your existing nat rules here...
iifname virbr0 ip daddr { 224.0.0.0/24, 255.255.255.255/32 } return \\\
    comment \"don't masquerade to reserved address blocks\"
iifname virbr0 oifname != virbr0 masquerade \\\
    comment \"masquerade all outgoing traffic from VMs\"
}
}
""))))

```

Ensure that you have IPv4 forwarding enabled (you can use `sysctl-service-type` for this).

5.4

- The (unofficial) libvirt Networking Handbook (<https://jamielinux.com/docs/libvirt-networking-handbook/index.html>)
Note that this resource is rather outdated at the time of writing (as of March 2025, it was last updated in 2015). Nevertheless, the authors of this chapter have found it to be a valuable source of information.

6

Guix is a functional package manager that offers many features beyond what more traditional package managers can do. To the uninitiated, those features might not have obvious use cases at first. The purpose of this chapter is to demonstrate some advanced package management concepts.

see Section “Package Management” in *GNU Guix Reference Manual* for a complete reference.

6.1 Guix

Guix provides a very useful feature that may be quite foreign to newcomers: *profiles*. They are a way to group package installations together and all users on the same system are free to use as many profiles as they want.

Whether you’re a developer or not, you may find that multiple profiles bring you great power and flexibility. While they shift the paradigm somewhat compared to *traditional package managers*, they are very convenient to use once you’ve understood how to set them up.

: This section is an opinionated guide on the use of multiple profiles. It predates `guix shell` and its fast profile cache (see Section “Invoking guix shell” in *GNU Guix Reference Manual*).

In many cases, you may find that using `guix shell` to set up the environment you need, when you need it, is less work than maintaining a dedicated profile. Your call!

If you are familiar with Python’s ‘`virtualenv`’, you can think of a profile as a kind of universal ‘`virtualenv`’ that can hold any kind of software whatsoever, not just Python software. Furthermore, profiles are self-sufficient: they capture all the runtime dependencies which guarantees that all programs within a profile will always work at any point in time.

Multiple profiles have many benefits:

- Clean semantic separation of the various packages a user needs for different contexts.
- Multiple profiles can be made available into the environment either on login or within a dedicated shell.
- Profiles can be loaded on demand. For instance, the user can use multiple shells, each of them running different profiles.
- Isolation: Programs from one profile will not use programs from the other, and the user can even install different versions of the same programs to the two profiles without conflict.
- Deduplication: Profiles share dependencies that happens to be the exact same. This makes multiple profiles storage-efficient.
- Reproducible: when used with declarative manifests, a profile can be fully specified by the Guix commit that was active when it was set up. This means that the exact same profile can be set up anywhere and anytime (<https://guix.gnu.org/blog/2018/multi-dimensional-transactions-and-rollbacks-oh-my/>), with just the commit information. See the section on Section 6.1.5 [], page 72.

- Easier upgrades and maintenance: Multiple profiles make it easy to keep package listings at hand and make upgrades completely frictionless.

Concretely, here follows some typical profiles:

- The dependencies of a project you are working on.
- Your favourite programming language libraries.
- Laptop-specific programs (like ‘`powertop`’) that you don’t need on a desktop.
- `TeXlive` (this one can be really useful when you need to install just one package for this one document you’ve just received over email).
- .

Let’s dive in the set up!

6.1.1

A Guix profile can be set up *via a manifest*. A manifest is a snippet of Scheme code that specifies the set of packages you want to have in your profile; it looks like this:

```
(specifications->manifest
  ("package-1"
   ;; Version 1.3 of package-2.
   "package-2@1.3"
   ;; The "lib" output of package-3.
   "package-3:lib"
   ; ...
   "package-N"))
```

See Section “Writing Manifests” in *GNU Guix Reference Manual*, for more information about the syntax.

We can create a manifest specification per profile and install them this way:

```
GUIX_EXTRA_PROFILES=$HOME/.guix-extra-profiles
mkdir -p "$GUIX_EXTRA_PROFILES"/my-project # if it does not exist yet
guix package --manifest=/path/to/guix-my-project-manifest.scm \
  --profile="$GUIX_EXTRA_PROFILES"/my-project/my-project
```

Here we set an arbitrary variable ‘`GUIX_EXTRA_PROFILES`’ to point to the directory where we will store our profiles in the rest of this article.

Placing all your profiles in a single directory, with each profile getting its own sub-directory, is somewhat cleaner. This way, each sub-directory will contain all the symlinks for precisely one profile. Besides, “looping over profiles” becomes obvious from any programming language (e.g. a shell script) by simply looping over the sub-directories of ‘`$GUIX_EXTRA_PROFILES`’.

Note that it’s also possible to loop over the output of

```
guix package --list-profiles
```

although you’ll probably have to filter out `~/.config/guix/current`.

To enable all profiles on login, add this to your `~/.bash_profile` (or similar):

```
for i in $GUIX_EXTRA_PROFILES/*; do
  profile=$i/$(basename "$i")
```

```

if [ -f "$profile"/etc/profile ]; then
    GUIX_PROFILE="$profile"
    . "$GUIX_PROFILE"/etc/profile
fi
unset profile
done

```

Note to Guix System users: the above reflects how your default profile `~/.guix-profile` is activated from `/etc/profile`, that latter being loaded by `~/.bashrc` by default.

You can obviously choose to only enable a subset of them:

```

for i in "$GUIX_EXTRA_PROFILES"/my-project-1 "$GUIX_EXTRA_PROFILES"/my-project-2; do
    profile=$i/$(basename "$i")
    if [ -f "$profile"/etc/profile ]; then
        GUIX_PROFILE="$profile"
        . "$GUIX_PROFILE"/etc/profile
    fi
    unset profile
done

```

When a profile is off, it's straightforward to enable it for an individual shell without "polluting" the rest of the user session:

```
GUIX_PROFILE="path/to/my-project" ; . "$GUIX_PROFILE"/etc/profile
```

The key to enabling a profile is to *source* its `'etc/profile'` file. This file contains shell code that exports the right environment variables necessary to activate the software contained in the profile. It is built automatically by Guix and meant to be sourced. It contains the same variables you would get if you ran:

```
guix package --search-paths=prefix --profile=$my_profile"
```

Once again, see Section “Invoking guix package” in *GNU Guix Reference Manual* for the command line options.

To upgrade a profile, simply install the manifest again:

```
guix package -m /path/to/guix-my-project-manifest.scm \
-p "$GUIX_EXTRA_PROFILES"/my-project/my-project
```

To upgrade all profiles, it's easy enough to loop over them. For instance, assuming your manifest specifications are stored in `~/.guix-manifests/guix-$profile-manifest.scm`, with `'$profile'` being the name of the profile (e.g. "project1"), you could do the following in Bourne shell:

```

for profile in "$GUIX_EXTRA_PROFILES"/*; do
    guix package --profile="$profile" \
        --manifest="$HOME/.guix-manifests/guix-$profile-manifest.scm"
done

```

Each profile has its own generations:

```
guix package -p "$GUIX_EXTRA_PROFILES"/my-project/my-project --list-generations
```

You can roll-back to any generation of a given profile:

```
guix package -p "$GUIX_EXTRA_PROFILES"/my-project/my-project --switch-generations=17
```

Finally, if you want to switch to a profile without inheriting from the current environment, you can activate it from an empty shell:

```
env -i $(which bash) --login --noprofile --norc
. my-project/etc/profile
```

6.1.2

Activating a profile essentially boils down to exporting a bunch of environmental variables. This is the role of the ‘etc/profile’ within the profile.

Note: Only the environmental variables of the packages that consume them will be set.

For instance, ‘MANPATH’ won’t be set if there is no consumer application for man pages within the profile. So if you need to transparently access man pages once the profile is loaded, you’ve got two options:

- Either export the variable manually, e.g.

```
export MANPATH=/path/to/profile${MANPATH:+:}$MANPATH
```
- Or include ‘man-db’ to the profile manifest.

The same is true for ‘INFOPATH’ (you can install ‘info-reader’), ‘PKG_CONFIG_PATH’ (install ‘pkg-config’), etc.

6.1.3 -

What about the default profile that Guix keeps in ~/.guix-profile?

You can assign it the role you want. Typically you would install the manifest of the packages you want to use all the time.

Alternatively, you could keep it “manifest-less” for throw-away packages that you would just use for a couple of days. This way makes it convenient to run

```
guix install package-foo
guix upgrade package-bar
```

without having to specify the path to a profile.

6.1.4

Manifests let you *declare* the set of packages you’d like to have in a profile (see Section “Writing Manifests” in *GNU Guix Reference Manual*). They are a convenient way to keep your package lists around and, say, to synchronize them across multiple machines using a version control system.

A common complaint about manifests is that they can be slow to install when they contain large number of packages. This is especially cumbersome when you just want get an upgrade for one package within a big manifest.

This is one more reason to use multiple profiles, which happen to be just perfect to break down manifests into multiple sets of semantically connected packages. Using multiple, small profiles provides more flexibility and usability.

Manifests come with multiple benefits. In particular, they ease maintenance:

- When a profile is set up from a manifest, the manifest itself is self-sufficient to keep a “package listing” around and reinstall the profile later or on a different system. For ad-hoc profiles, we would need to generate a manifest specification manually and maintain the package versions for the packages that don’t use the default version.

- `guix package --upgrade` always tries to update the packages that have propagated inputs, even if there is nothing to do. Guix manifests remove this problem.
- When partially upgrading a profile, conflicts may arise (due to diverging dependencies between the updated and the non-updated packages) and they can be annoying to resolve manually. Manifests remove this problem altogether since all packages are always upgraded at once.
- As mentioned above, manifests allow for reproducible profiles, while the imperative `guix install`, `guix upgrade`, etc. do not, since they produce different profiles every time even when they hold the same packages. See the related discussion on the matter (<https://issues.guix.gnu.org/issue/33285>).
- Manifest specifications are usable by other ‘guix’ commands. For example, you can run `guix weather -m manifest.scm` to see how many substitutes are available, which can help you decide whether you want to try upgrading today or wait a while. Another example: you can run `guix pack -m manifest.scm` to create a pack containing all the packages in the manifest (and their transitive references).
- Finally, manifests have a Scheme representation, the ‘<manifest>’ record type. They can be manipulated in Scheme and passed to the various Guix APIs (<https://en.wikipedia.org/wiki/Api>).

It’s important to understand that while manifests can be used to declare profiles, they are not strictly equivalent: profiles have the side effect that they “pin” packages in the store, which prevents them from being garbage-collected (see Section “Invoking `guix gc`” in *GNU Guix Reference Manual*) and ensures that they will still be available at any point in the future. The `guix shell` command also protects recently-used profiles from garbage collection; profiles that have not been used for a while may be garbage-collected though, along with the packages they refer to.

To be 100% sure that a given profile will never be collected, install the manifest to a profile and use `GUIX_PROFILE=/the/profile; . "$GUIX_PROFILE"/etc/profile` as explained above: this guarantees that our hacking environment will be available at all times.

Security warning: While keeping old profiles around can be convenient, keep in mind that outdated packages may not have received the latest security fixes.

6.1.5

To reproduce a profile bit-for-bit, we need two pieces of information:

- a manifest (see Section “Writing Manifests” in *GNU Guix Reference Manual*);
- a Guix channel specification (see Section “Replicating Guix” in *GNU Guix Reference Manual*).

Indeed, manifests alone might not be enough: different Guix versions (or different channels) can produce different outputs for a given manifest.

You can output the Guix channel specification with ‘`guix describe --format=channels`’ (see Section “Invoking `guix describe`” in *GNU Guix Reference Manual*). Save this to a file, say ‘`channel-specs.scm`’.

On another computer, you can use the channel specification file and the manifest to reproduce the exact same profile:

```
GUIX_EXTRA_PROFILES=$HOME/.guix-extra-profiles
```

```
GUIX_EXTRA=$HOME/.guix-extra

mkdir -p "$GUIX_EXTRA"/my-project
guix pull --channels=channel-specs.scm --profile="$GUIX_EXTRA/my-project/guix"

mkdir -p "$GUIX_EXTRA_PROFILES/my-project"
"$GUIX_EXTRA"/my-project/guix/bin/guix package \
  --manifest=/path/to/guix-my-project-manifest.scm \
  --profile="$GUIX_EXTRA_PROFILES"/my-project/my-project
```

It's safe to delete the Guix channel profile you've just installed with the channel specification, the project profile does not depend on it.

7

Guix is a handy tool for developers; `guix shell`, in particular, gives a standalone development environment for your package, no matter what language(s) it’s written in (see Section “Invoking guix shell” in *GNU Guix Reference Manual*). To benefit from it, you have to initially write a package definition and have it either in Guix proper, or in a channel, or directly in your project’s source tree as a `guix.scm` file. This last option is appealing: all developers have to do to get set up is clone the project’s repository and run `guix shell`, with no arguments.

Development needs go beyond development environments though. How can developers perform continuous integration of their code in Guix build environments? How can they deliver their code straight to adventurous users? This chapter describes a set of files developers can add to their repository to set up Guix-based development environments, continuous integration, and continuous delivery—all at once¹.

7.1

How do we go about “Guixifying” a repository? The first step, as we’ve seen, will be to add a `guix.scm` at the root of the repository in question. We’ll take Guile (<https://www.gnu.org/software/guile>) as an example in this chapter: it’s written in Scheme (mostly) and C, and has a number of dependencies—a C compilation tool chain, C libraries, Autoconf and its friends, LaTeX, and so on. The resulting `guix.scm` looks like the usual package definition (see Section “Defining Packages” in *GNU Guix Reference Manual*), just without the `define-public` bit:

```
;; The ‘guix.scm’ file for Guile, for use by ‘guix shell’.
```

```
(use-modules (guix)
             (guix build-system gnu)
             ((guix licenses) #:prefix license:)
             (gnu packages autotools)
             (gnu packages base)
             (gnu packages bash)
             (gnu packages bdw-gc)
             (gnu packages compression)
             (gnu packages flex)
             (gnu packages gdb)
             (gnu packages gettext)
             (gnu packages gperf)
             (gnu packages libffi)
             (gnu packages libunistring)
             (gnu packages linux)
             (gnu packages pkg-config)
             (gnu packages readline))
```

¹ This chapter is adapted from a blog post (<https://guix.gnu.org/en/blog/2023/from-development-environments-to-continuous-integrationthe-ultimate-guide-to-software-development-with-guix/>) published in June 2023 on the Guix web site.

```

(gnu packages tex)
(gnu packages texinfo)
(gnu packages version-control))

(package
  (name "guile")
  (version "3.0.99-git") ;funky version number
  (source #f) ;no source
  (build-system gnu-build-system)
  (native-inputs
    (append (list autoconf
                  automake
                  libtool
                  gnu-gettext
                  flex
                  texinfo
                  texlive-base ;for "make pdf"
                  texlive-epsf
                  gperf
                  git
                  gdb
                  strace
                  readline
                  lzip
                  pkg-config)

              ;; When cross-compiling, a native version of Guile itself is
              ;; needed.
              (if (%current-target-system)
                  (list this-package)
                  '()))))
  (inputs
    (list libffi bash-minimal))
  (propagated-inputs
    (list libunistring libgc))

  (native-search-paths
    (list (search-path-specification
            (variable "GUILE_LOAD_PATH")
            (files '("share/guile/site/3.0"))))
          (search-path-specification
            (variable "GUILE_LOAD_COMPILED_PATH")
            (files '("lib/guile/3.0/site-ccache")))))
  (synopsis "Scheme implementation intended especially for extensions")
  (description
    "Guile is the GNU Ubiquitous Intelligent Language for Extensions,
    and it's actually a full-blown Scheme implementation!")

```



```
(home-page "https://www.gnu.org/software/guile/")
(license license:lgpl3+))
```

Quite a bit of boilerplate, but now someone who'd like to hack on Guile now only needs to run:

```
guix shell
```

That gives them a shell containing all the dependencies of Guile: those listed above, but also *implicit dependencies* such as the GCC tool chain, GNU Make, sed, grep, and so on. See Section “Invoking guix shell” in *GNU Guix Reference Manual*, for more info on `guix shell`.

The chef’s recommendation: Our suggestion is to create development environments like this:

```
guix shell --container --link-profile
```

... or, for short:

```
guix shell -CP
```

That gives a shell in an isolated container, and all the dependencies show up in `$HOME/.guix-profile`, which plays well with caches such as `config.cache` (see Section “Cache Files” in *Autoconf*) and absolute file names recorded in generated `Makefiles` and the likes. The fact that the shell runs in a container brings peace of mind: nothing but the current directory and Guile’s dependencies is visible inside the container; nothing from the system can possibly interfere with your development.

7.2 Level 1: Building with Guix

Now that we have a package definition (see Section 7.1 [], page 74), why not also take advantage of it so we can build Guile with Guix? We had left the `source` field empty, because `guix shell` above only cares about the *inputs* of our package—so it can set up the development environment—not about the package itself.

To build the package with Guix, we’ll need to fill out the `source` field, along these lines:

```
(use-modules (guix)
             (guix git-download) ;for ‘git-predicate’
             ...)

(define vcs-file?
  ;; Return true if the given file is under version control.
  (or (git-predicate (current-source-directory))
      (const #t))) ;not in a Git checkout

(package
  (name "guile")
  (version "3.0.99-git") ;funky version number
  (source (local-file "." "guile-checkout"
                     #:recursive? #t
                     #:select? vcs-file?))
  ...)
```

Here’s what we changed compared to the previous section:

1. We added (`guix git-download`) to our set of imported modules, so we can use its `git-predicate` procedure.
2. We defined `vcs-file?` as a procedure that returns true when passed a file that is under version control. For good measure, we add a fallback case for when we’re not in a Git checkout: always return true.
3. We set `source` to a `local-file` (https://guix.gnu.org/manual/devel/en/html_node/G_002dExpressions.html#index-local_002dfile)—a recursive copy of the current directory (`"."`), limited to files under version control (the `#:select?` bit).

From there on, our `guix.scm` file serves a second purpose: it lets us build the software with Guix. The whole point of building with Guix is that it’s a “clean” build—you can be sure nothing from your working tree or system interferes with the build result—and it lets you test a variety of things. First, you can do a plain native build:

```
guix build -f guix.scm
```

But you can also build for another system (possibly after setting up see Section “Daemon Offload Setup” in *GNU Guix Reference Manual* or see Section “Virtualization Services” in *GNU Guix Reference Manual*):

```
guix build -f guix.scm -s aarch64-linux -s riscv64-linux
```

... or cross-compile:

```
guix build -f guix.scm --target=x86_64-w64-mingw32
```

You can also use *package transformations* to test package variants (see Section “Package Transformation Options” in *GNU Guix Reference Manual*):

```
# What if we built with Clang instead of GCC?
guix build -f guix.scm \
  --with-c-toolchain=guile@3.0.99-git=clang-toolchain
```

```
# What about that under-tested configure flag?
guix build -f guix.scm \
  --with-configure-flag=guile@3.0.99-git=--disable-networking
```

Handy!

7.3 Level 2: The Repository as a Channel

We now have a Git repository containing (among other things) a package definition (see Section 7.2 [Guix], page 76). Can’t we turn it into a *channel* (see Section “Channels” in *GNU Guix Reference Manual*)? After all, channels are designed to ship package definitions to users, and that’s exactly what we’re doing with our `guix.scm`.

Turns out we can indeed turn it into a channel, but with one caveat: we must create a separate directory for the `.scm` file(s) of our channel so that `guix pull` doesn’t load unrelated `.scm` files when someone pulls the channel—and in Guile, there are lots of them! So we’ll start like this, keeping a top-level `guix.scm` symlink for the sake of `guix shell`:

```
mkdir -p .guix/modules
mv guix.scm .guix/modules/guile-package.scm
ln -s .guix/modules/guile-package.scm guix.scm
```

To make it usable as part of a channel, we need to turn our `guix.scm` file into a *package module* (see Section “Package Modules” in *GNU Guix Reference Manual*): we do that by changing the `use-modules` form at the top to a `define-module` form. We also need to actually *export* a package variable, with `define-public`, while still returning the package value at the end of the file so we can still use `guix shell` and `guix build -f guix.scm`. The end result looks like this (not repeating things that haven’t changed):

```
(define-module (guile-package)
  #:use-module (guix)
  #:use-module (guix git-download) ;for ‘git-predicate’
  ...)

(define vcs-file?
  ;; Return true if the given file is under version control.
  (or (git-predicate (dirname (dirname (current-source-directory))))
      (const #t))) ;not in a Git checkout

(define-public guile
  (package
    (name "guile")
    (version "3.0.99-git") ;funky version number
    (source (local-file "../.." "guile-checkout"
                       #:recursive? #t
                       #:select? vcs-file?))
    ...))

;; Return the package object define above at the end of the module.
guile
```

We need one last thing: a `.guix-channel` file (https://guix.gnu.org/manual/devel/en/html_node/Package-Modules-in-a-Sub_002ddirectory.html) so Guix knows where to look for package modules in our repository:

```
;; This file lets us present this repo as a Guix channel.

(channel
  (version 0)
  (directory ".guix/modules")) ;look for package modules under .guix/modules/
```

To recap, we now have these files:

```
.
.guix-channel
guix.scm → .guix/modules/guile-package.scm
.guix
  modules
    guile-package.scm
```

And that’s it: we have a channel! (We could do better and support *channel authentication* (https://guix.gnu.org/manual/devel/en/html_node/Specifying-Channel-Authorizations.html) so users know they’re pulling genuine

code. We'll spare you the details here but it's worth considering!) Users can pull from this channel by adding it to `~/.config/guix/channels.scm` (https://guix.gnu.org/manual/devel/en/html_node/Specifying-Additional-Channels.html), along these lines:

```
(append (list (channel
               (name 'guile)
               (url "https://git.savannah.gnu.org/git/guile.git")
               (branch "main"))))
      %default-channels)
```

After running `guix pull`, we can see the new package:

```
$ guix describe
Generation 264 May 26 2023 16:00:35 (current)
guile 36fd2b4
  repository URL: https://git.savannah.gnu.org/git/guile.git
  branch: main
  commit: 36fd2b4920ae926c79b936c29e739e71a6dff2bc
guix c5bc698
  repository URL: https://git.guix.gnu.org/guix.git
  commit: c5bc698e8922d78ed85989985cc2ceb034de2f23
$ guix package -A ^guile$
guile 3.0.99-git out,debug guile-package.scm:51:4
guile 3.0.9 out,debug gnu/packages/guile.scm:317:2
guile 2.2.7 out,debug gnu/packages/guile.scm:258:2
guile 2.2.4 out,debug gnu/packages/guile.scm:304:2
guile 2.0.14 out,debug gnu/packages/guile.scm:148:2
guile 1.8.8 out gnu/packages/guile.scm:77:2
$ guix build guile@3.0.99-git
[...]
/gnu/store/axnzbl89yz7ld78bmx72vpqp802dwsar-guile-3.0.99-git-debug
/gnu/store/r34gsij7f0glg2fbakcmmk0zn4v62s5w-guile-3.0.99-git
```

That's how, as a developer, you get your software delivered directly into the hands of users! No intermediaries, yet no loss of transparency and provenance tracking.

With that in place, it also becomes trivial for anyone to create Docker images, Deb/RPM packages, or a plain tarball with `guix pack` (see Section “Invoking `guix pack`” in *GNU Guix Reference Manual*):

```
# How about a Docker image of our Guile snapshot?
guix pack -f docker -S /bin=bin guile@3.0.99-git

# And a relocatable RPM?
guix pack -f rpm -R -S /bin=bin guile@3.0.99-git
```

7.4 Bonus: Package Variants

We now have an actual channel, but it contains only one package (see Section 7.3 [], page 77). While we're at it, we can define *package variants* (see Section “Defining Package Variants” in *GNU Guix Reference Manual*) in our `guile-package.scm` file, variants

that we want to be able to test as Guile developers—similar to what we did above with transformation options. We can add them like so:

```
;; This is the '.guix/modules/guile-package.scm' file.

(define-module (guile-package)
  ...)

(define-public guile
  ...)

(define (package-with-configure-flags p flags)
  "Return P with FLAGS as additional 'configure' flags."
  (package/inherit p
    (arguments
      (substitute-keyword-arguments (package-arguments p)
        ((#:configure-flags original-flags #~(list))
         #~(append #$original-flags #$flags))))))

(define-public guile-without-threads
  (package
    (inherit (package-with-configure-flags guile
      #~(list "--without-threads"))))
    (name "guile-without-threads")))

(define-public guile-without-networking
  (package
    (inherit (package-with-configure-flags guile
      #~(list "--disable-networking"))))
    (name "guile-without-networking")))

;; Return the package object defined above at the end of the module.
guile
```

We can build these variants as regular packages once we've pulled the channel. Alternatively, from a checkout of Guile, we can run a command like this one from the top level:

```
guix build -L $PWD/.guix/modules guile-without-threads
```

7.5 Level 3: Setting Up Continuous Integration

The channel we defined above (see Section 7.3 [], page 77) becomes even more interesting once we set up *continuous integration* (https://en.wikipedia.org/wiki/Continuous_integration) (CI). There are several ways to do that.

You can use one of the mainstream continuous integration tools, such as GitLab-CI. To do that, you need to make sure you run jobs in a Docker image or virtual machine that has

Guix installed. If we were to do that in the case of Guile, we’d have a job that runs a shell command like this one:

```
guix build -L $PWD/.guix/modules guile@3.0.99-git
```

Doing this works great and has the advantage of being easy to achieve on your favorite CI platform.

That said, you’ll really get the most of it by using Cuirass (<https://guix.gnu.org/en/cuirass>), a CI tool designed for and tightly integrated with Guix. Using it is more work than using a hosted CI tool because you first need to set it up, but that setup phase is greatly simplified if you use its Guix System service (see Section “Continuous Integration” in *GNU Guix Reference Manual*). Going back to our example, we give Cuirass a spec file that goes like this:

```
;; Cuirass spec file to build all the packages of the ‘guile’ channel.
(list (specification
      (name "guile")
      (build '(channels guile))
      (channels
        (append (list (channel
                       (name 'guile)
                       (url "https://git.savannah.gnu.org/git/guile.git")
                       (branch "main"))))
              %default-channels))))
```

It differs from what you’d do with other CI tools in two important ways:

- Cuirass knows it’s tracking *two* channels, **guile** and **guix**. Indeed, our own **guile** package depends on many packages provided by the **guix** channel—GCC, the GNU **libc**, **libffi**, and so on. Changes to packages from the **guix** channel can potentially influence our **guile** build and this is something we’d like to see as soon as possible as Guile developers.
- Build results are not thrown away: they can be distributed as *substitutes* so that users of our **guile** channel transparently get pre-built binaries! (see Section “Substitutes” in *GNU Guix Reference Manual*, for background info on substitutes.)

From a developer’s viewpoint, the end result is this status page (<https://ci.guix.gnu.org/jobset/guile>) listing *evaluations*: each evaluation is a combination of commits of the **guix** and **guile** channels providing a number of *jobs*—one job per package defined in **guile-package.scm** times the number of target architectures.

As for substitutes, they come for free! As an example, since our **guile** jobset is built on ci.guix.gnu.org, which runs **guix publish** (see Section “Invoking guix publish” in *GNU Guix Reference Manual*) in addition to Cuirass, one automatically gets substitutes for **guile** builds from ci.guix.gnu.org; no additional work is needed for that.

7.6 Bonus: Build manifest

The Cuirass spec above is convenient: it builds every package in our channel, which includes a few variants (see Section 7.5 [], page 80). However, this might be insufficiently expressive in some cases: one might want specific cross-compilation jobs, transformations, Docker images, RPM/Deb packages, or even system tests.

To achieve that, you can write a *manifest* (see Section “Writing Manifests” in *GNU Guix Reference Manual*). The one we have for Guile has entries for the package variants we defined above, as well as additional variants and cross builds:

```
;; This is '.guix/manifest.scm'.

(use-modules (guix)
             (guix profiles)
             (guile-package)) ;import our own package module

(define* (package->manifest-entry* package system
        #:key target)
  "Return a manifest entry for PACKAGE on SYSTEM, optionally cross-compiled to
  TARGET."
  (manifest-entry
   (inherit (package->manifest-entry package))
   (name (string-append (package-name package) "." system
                        (if target
                            (string-append "." target)
                            "")))
   (item (with-parameters ((%current-system system)
                          (%current-target-system target))
          package))))

(define native-builds
  (manifest
   (append (map (lambda (system)
                  (package->manifest-entry* guile system))

                '("x86_64-linux" "i686-linux"
                  "aarch64-linux" "armhf-linux"
                  "powerpc64le-linux")))
   (map (lambda (guile)
          (package->manifest-entry* guile "x86_64-linux"))
        (cons (package
                 (inherit (package-with-c-toolchain
                          guile
                          `(("clang-toolchain"
                            ,(specification->package
                               "clang-toolchain")))))
                 (name "guile-clang"))
                (list guile-without-threads
                      guile-without-networking
                      guile-debug
                      guile-strict-typing))))))

(define cross-builds
```

```
(manifest
  (map (lambda (target)
        (package->manifest-entry* guile "x86_64-linux"
                                     #:target target))

    '("i586-pc-gnu"
      "aarch64-linux-gnu"
      "riscv64-linux-gnu"
      "i686-w64-mingw32"
      "x86_64-linux-gnu"))))

(concatenate-manifests (list native-builds cross-builds))
```

We won't go into the details of this manifest; suffice to say that it provides additional flexibility. We now need to tell Cuirass to build this manifest, which is done with a spec slightly different from the previous one:

```
;; Cuirass spec file to build all the packages of the 'guile' channel.
(list (specification
      (name "guile")
      (build '(manifest ".guix/manifest.scm"))
      (channels
        (append (list (channel
                        (name 'guile)
                        (url "https://git.savannah.gnu.org/git/guile.git")
                        (branch "main"))
                      %default-channels))))))
```

We changed the (build ...) part of the spec to '(manifest ".guix/manifest.scm") so that it would pick our manifest, and that's it!

7.7

We picked Guile as the running example in this chapter and you can see the result here:

- `.guix-channel` (<https://git.savannah.gnu.org/cgit/guile.git/tree/.guix-channel?id=cd57379b3df636198d8cd8e76c1bfbc523762e79>);
- `.guix/modules/guile-package.scm` (<https://git.savannah.gnu.org/cgit/guile.git/tree/.guix/modules/guile-package.scm?id=cd57379b3df636198d8cd8e76c1bfbc523762e79>) with the top-level `guix.scm` symlink;
- `.guix/manifest.scm` (<https://git.savannah.gnu.org/cgit/guile.git/tree/.guix/manifest.scm?id=cd57379b3df636198d8cd8e76c1bfbc523762e79>).

These days, repositories are commonly peppered with dot files for various tools: `.envrc`, `.gitlab-ci.yml`, `.github/workflows`, `Dockerfile`, `.buildpacks`, `Aptfile`, `requirements.txt`, and `whatnot`. It may sound like we're proposing a bunch of *additional* files, but in fact those files are expressive enough to *supersede* most or all of those listed above.

With a couple of files, we get support for:

- development environments (`guix shell`);

- pristine test builds, including for package variants and for cross-compilation (`guix build`);
- continuous integration (with Cuirass or with some other tool);
- continuous delivery to users (*via* the channel and with pre-built binaries);
- generation of derivative build artifacts such as Docker images or Deb/RPM packages (`guix pack`).

This a nice (in our view!) unified tool set for reproducible software deployment, and an illustration of how you as a developer can benefit from it!

8

Guix provides multiple tools to manage environment. This chapter demonstrate such utilities.

8.1 Guix direnv

Guix provides a ‘`direnv`’ package, which could extend shell after directory change. This tool could be used to prepare a pure Guix environment.

The following example provides a shell function for `~/.direnvrc` file, which could be used from Guix Git repository in `~/src/guix/.envrc` file to setup a build environment similar to described in see Section “Building from Git” in *GNU Guix Reference Manual*.

Create a `~/direnvrc` with a Bash code:

```
# Thanks <https://github.com/direnv/direnv/issues/73#issuecomment-152284914>
export_function()
{
    local name=$1
    local alias_dir=$PWD/.direnv/aliases
    mkdir -p "$alias_dir"
    PATH_add "$alias_dir"
    local target="$alias_dir/$name"
    if declare -f "$name" >/dev/null; then
        echo "##$SHELL" > "$target"
        declare -f "$name" >> "$target" 2>/dev/null
        # Notice that we add shell variables to the function trigger.
        echo "$name \$*" >> "$target"
        chmod +x "$target"
    fi
}

use_guix()
{
    # Set GitHub token.
    export GUIX_GITHUB_TOKEN="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

    # Unset 'GUIX_PACKAGE_PATH'.
    export GUIX_PACKAGE_PATH=""

    # Recreate a garbage collector root.
    gcroots="$HOME/.config/guix/gcroots"
    mkdir -p "$gcroots"
    gcroot="$gcroots/guix"
    if [ -L "$gcroot" ]
    then
        rm -v "$gcroot"
    fi
}
```

```

# Miscellaneous packages.
PACKAGES_MAINTENANCE=(
    direnv
    git
    git:send-email
    git-cal
    gnupg
    guile-colored
    guile-readline
    less
    ncurses
    openssh
    xdot
)

# Environment packages.
PACKAGES=(help2man guile-sqlite3 guile-gcrypt)

# Thanks <https://lists.gnu.org/archive/html/guix-devel/2016-09/msg00859.html>
eval "$(guix shell --search-paths --root="$gcroot" --pure \
    --development guix ${PACKAGES[@]} ${PACKAGES_MAINTENANCE[@]} "$@" )"

# Predefine configure flags.
configure()
{
    ./configure
}
export_function configure

# Run make and optionally build something.
build()
{
    make -j 2
    if [ $# -gt 0 ]
    then
        ./pre-inst-env guix build "$@"
    fi
}
export_function build

# Predefine push Git command.
push()
{
    git push --set-upstream origin
}
export_function push

```

```
clear                # Clean up the screen.
git-cal --author='Your Name' # Show contributions calendar.

# Show commands help.
echo "
build          build a package or just a project if no argument provided
configure      run ./configure with predefined parameters
push           push to upstream Git repository
"
}
```

Every project containing `.envrc` with a string `use guix` will have predefined environment variables and procedures.

Run `direnv allow` to setup the environment for the first time.

9 Using Guix for Reproducible Research

Because it supports reproducible deployment, Guix is a solid foundation for *reproducible research workflows*. This section is targeted at scientists; it shows how to add Guix to one’s reproducible research toolbox¹.

With Guix as the basis of your computational workflow, you can get what’s in essence *executable provenance meta-data*: it’s like the list of package name/version pairs some provide as an appendix to their publication, except more precise and immediately deployable.

This section is a guide in just four steps on how to make your computational experiments reproducible using Guix, and how to provide that information in your research paper.

9.1 Step 1: Setting Up the Environment

The first step is to identify precisely what packages you need in your software environment to run your computational experiment.

Assuming you have a Python script that uses NumPy, you can start by creating an environment that contains these two packages and to run your code in that environment (see Section “Invoking guix shell” in *GNU Guix Reference Manual*):

```
guix shell -C python python-numpy -- python3 ./myscript.py
```

The `-C` flag here (or `--container`) instructs `guix shell` to create that environment in an isolated container with nothing but the two packages you asked for. That way, if `./myscript.py` needs more than these two packages, it’ll fail to run and you’ll immediately notice. On some systems `--container` is not supported; in that case, you can resort to `--pure` instead.

Perhaps you’ll find that you also need Pandas and add it to the environment:

```
guix shell -C python python-numpy python-pandas -- \
python3 ./myscript.py
```

If you fail to guess the name of the package (this one was easy!), try `guix search`.

Environments for Python, R, and similar high-level languages are relatively easy to set up. For C/C++ code, you may need more packages:

```
guix shell -C gcc-toolchain cmake coreutils grep sed make -- ...
```

Or perhaps you’ll find that you could just as well provide a package definition for your code—see Section “Defining Packages” in *GNU Guix Reference Manual*, to learn more on how to do that.

Eventually, you’ll have a list of packages that satisfies your needs.

What if a package is missing?: Guix and the main scientific channels provide about tens of thousands of packages (<https://hpc.guix.info/browse>). Yet, there’s always the possibility that the one package you need is missing.

In that case, you will need to provide a definition for it (see Section “Defining Packages” in *GNU Guix Reference Manual*) in a dedicated channel of yours (see Section “Creating a Channel” in *GNU Guix Reference Manual*). For software

¹ This chapter is adapted from a blog post published on the Guix-HPC web site in 2023. (<https://hpc.guix.info/blog/2023/06/a-guide-to-reproducible-research-papers/>).

in Python, R, and other high-level languages, most of the work can usually be automated by using `guix import` (see Section “Invoking `guix import`” in *GNU Guix Reference Manual*).

Join the friendly Guix community (<https://guix.gnu.org/contact/>) to get help!

9.2 Step 2: Recording the Environment

Now that you have that `guix shell` command line with a list of packages, the best course of action is to save it in a *manifest* file—essentially a software bill of materials—that Guix can then ingest (see Section “Writing Manifests” in *GNU Guix Reference Manual*). The easiest way to get started is by “translating” your command line into a manifest:

```
guix shell python python-numpy python-pandas \
--export-manifest > manifest.scm
```

Put that manifest under version control! From there anyone can redeploy the software environment described by the manifest and run code in that environment:

```
guix shell -C -m manifest.scm -- python3 ./myscript.py
```

Here’s what `manifest.scm` reads:

```
;; What follows is a "manifest" equivalent to the command line you gave.
;; You can store it in a file that you may then pass to any 'guix' command
;; that accepts a '--manifest' (or '-m') option.
```

```
(specifications->manifest
 (list "python" "python-numpy" "python-pandas"))
```

It’s a code snippet that lists packages. Notice that there are no version numbers! Indeed, these version numbers are specified in package definitions, located in Guix channels. To allow others to reproduce the exact same environment as the one you’re running, you need to *pin Guix itself*, by capturing the current Guix channel commits with `guix describe` (see Section “Replicating Guix” in *GNU Guix Reference Manual*):

```
guix describe -f channels > channels.scm
```

This `channels.scm` file is similar in spirit to “lock files” that some deployment tools employ to pin package revisions. You should also keep it under version control in your code, and possibly update it once in a while when you feel like running your code against newer versions of its dependencies. With this file, anyone, *at any time and on any machine*, can now reproduce the exact same environment by running:

```
guix time-machine -C channels.scm -- \
  shell -C -m manifest.scm -- \
  python3 ./myscript.py
```

In this example we rely solely on the `guix` channel, which provides the Python packages we need. Perhaps some of the packages you need live in other channels (<https://hpc.guix.info/channels>)—maybe `guix-cran` if you use R, maybe `guix-science`. That’s fine: `guix describe` also captures that.

Of course do include a `README` file giving the exact command to run the code. Not everyone uses Guix so it can be helpful to also provide minimal non-Guix setup instructions: which package versions are used, how software is built, etc. As we have seen, such

instructions would likely be inaccurate and inconvenient to follow at best. Yet, it can be a useful starting point to someone trying to recreate a *similar* environment using different tools. It should probably be presented as such, with the understanding that the only way to get the *same* environment is to use Guix.

9.3 Step 3: Ensuring Long-Term Source Code Archiving

We insisted on version control before: for the `manifest.scm` and `channels.scm` files, but of course also for your own code. Our recommendation is to have these two `.scm` files in the same repository as the code they’re about.

Since the goal is enabling reproducibility, source code availability is a prime concern. Source code hosting services come and go and we don’t want our code to vanish in a whim and render our published research work unverifiable. Software Heritage (<https://www.softwareheritage.org/>) (SWH for short) is *the* solution for this: SWH archives public source code and provides unique intrinsic identifiers to refer to it—*SWHIDs* (<https://swhid.org>). Guix itself is connected to SWH (<https://doi.org/10.1145/3641525.3663622>) to (1) ensure that the source code of its packages is archived, and (2) to fall back to downloading from the SWH archive should code vanish from its original site.

Once your own code is available in a public version-control repository, such as a Git repository on your lab’s hosting service, you can ask SWH to archive it by going to its Save Code Now (<https://archive.softwareheritage.org/save/>) interface. SWH will process the request asynchronously and eventually you’ll find your code has made it into the archive (<https://archive.softwareheritage.org/>).

9.4 Step 4: Referencing the Software Environment

This brings us to the last step: referring to our code *and* software environment in our beloved paper. We already have all our code and Guix files in the same repository, which is archived on SWH. Thanks to SWH, we now have a SWHID, which uniquely identifies the relevant revision of our code.

Following SWH’s own guide (<https://www.softwareheritage.org/howto-archive-and-reference-your-code/>), we’ll pick an `swh:dir` kind of identifier, which refers to the directory of the relevant revision/commit of our repository, and we’ll keep *contextual info* for clarity—that includes the original URL. Putting it all together, we’ll conclude our paper with a sentence along these lines:

Example: The source code used to produce this study, as well as instructions to run it in the right software environment using GNU Guix, is archived on Software Heritage as `swh:1:dir:cc8919d7705fbba31efa677ce00bef7eb374fb80;origin=https://gitlab.inria.fr/lcourtes-phd/edcc-2006-redone;visit=swh:1:snp:71a4d08ef4a2e8455b67ef0c6b82349e82870b46;anchor=swh:1:dir:cc8919d7705fbba31efa677ce00bef7eb374fb80` (<https://archive.softwareheritage.org/swh:1:dir:cc8919d7705fbba31efa677ce00bef7eb374fb80>) (<https://gitlab.inria.fr/lcourtes-phd/edcc-2006-redone;visit=swh:1:snp:71a4d08ef4a2e8455b67ef0c6b82349e82870b46>).

With this information, the reader can:

- get the source code;
- reproduce its software environment with `guix time-machine` and run the code;
- inspect and possibly modify both the code and its environment.

Mission accomplished!

10 Installing Guix on a Cluster

Guix is appealing to scientists and HPC (high-performance computing) practitioners: it makes it easy to deploy potentially complex software stacks, and it lets you do so in a reproducible fashion—you can redeploy the exact same software on different machines and at different points in time.

In this chapter we look at how a cluster sysadmin can install Guix for system-wide use, such that it can be used on all the cluster nodes, and discuss the various tradeoffs¹.

: Here we assume that the cluster is running a GNU/Linux distro other than Guix System and that we are going to install Guix on top of it.

10.1

The recommended approach is to set up one *head node* running `guix-daemon` and exporting `/gnu/store` over NFS to compute nodes.

Remember that `guix-daemon` is responsible for spawning build processes and downloads on behalf of clients (see Section “Invoking guix-daemon” in *GNU Guix Reference Manual*), and more generally accessing `/gnu/store`, which contains all the package binaries built by all the users (see Section “The Store” in *GNU Guix Reference Manual*). “Client” here refers to all the Guix commands that users see, such as `guix install`. On a cluster, these commands may be running on the compute nodes and we’ll want them to talk to the head node’s `guix-daemon` instance.

To begin with, the head node can be installed following the usual binary installation instructions (see Section “Binary Installation” in *GNU Guix Reference Manual*). Thanks to the installation script, this should be quick. Once installation is complete, we need to make some adjustments.

: Starting from Guix version 1.5.0, `guix-daemon` runs without root privileges by default; previously installed systems can also be migrated to this “rootless” mode (see Section “Build Environment Setup” in *GNU Guix Reference Manual*). Learn more about this change in a 2025 blog post (<https://hpc.guix.info/blog/2025/03/build-daemon-drops-its-privileges/>).

Since we want `guix-daemon` to be reachable not just from the head node but also from the compute nodes, we need to arrange so that it listens for connections over TCP/IP. To do that, we’ll edit the systemd startup file for `guix-daemon`, `/etc/systemd/system/guix-daemon.service`, and add a `--listen` argument to the `ExecStart` line so that it looks something like this:

```
ExecStart=/var/guix/profiles/per-user/root/current-guix/bin/guix-daemon \
--listen=/var/guix/daemon-socket/socket --listen=0.0.0.0
```

For these changes to take effect, the service needs to be restarted:

```
systemctl daemon-reload
systemctl restart guix-daemon.service
```

: The `--listen=0.0.0.0` bit means that `guix-daemon` will process *all* incoming TCP connections on port 44146 (see Section “Invoking guix-daemon” in *GNU*

¹ This chapter is adapted from a blog post published on the Guix-HPC web site in 2017 (<https://hpc.guix.info/blog/2017/11/installing-guix-on-a-cluster/>).

Guix Reference Manual). This is usually fine in a cluster setup where the head node is reachable exclusively from the cluster’s local area network—you don’t want that to be exposed to the Internet!

The next step is to define our NFS exports in `/etc/exports` (<https://linux.die.net/man/5/exports>) by adding something along these lines:

```
/gnu/store    *(ro)
/var/guix     *(rw, async)
/var/log/guix *(ro)
```

The `/gnu/store` directory can be exported read-only since only `guix-daemon` on the master node will ever modify it. `/var/guix` contains *user profiles* as managed by `guix package`; thus, to allow users to install packages with `guix package`, this must be read-write.

: Although it is rarely used, Guix allows for the creation of profiles in non-default locations. For instance, `guix package -p ~/dev/python-dev -i python` installs Python in a profile reachable from the `~/dev/python-dev` symlink instead of the default `~/.guix-profile`.

To make sure that this profile is protected from garbage collection—i.e., that Python will not be removed from `/gnu/store` while this profile exists—, *home directories should be mounted on the head node* as well so that `guix-daemon` knows about these non-standard profiles and avoids collecting software they refer to.

Doing so is optional though: profiles created by default by `guix package`, `guix pull`, and `guix shell` are all stored under `/var/guix` and thus protected from garbage collection.

It may be a good idea to periodically remove unused bits from `/gnu/store` by running `guix gc` (see Section “Invoking `guix gc`” in *GNU Guix Reference Manual*). This can be done by adding a systemd timer based on the one shipped with Guix:

```
cp \
  /var/guix/profiles/per-user/root/current-guix/lib/systemd/system/guix-gc.* \
  /etc/systemd/system
```

Make sure to edit `guix-gc.service` and `guix-gc.timer` under `/etc/systemd/system` to adjust the parameters to your need: timer frequency, amount of free space, and whether to delete user’s old generations. We recommend executing `guix gc -F50G` every night, which will ensure that at least 50 GiB are available, without deleting user generations. Once you’re satisfied, you can start the timer:

```
systemctl daemon-reload
systemctl start guix-gc.timer
```

We’re done with the head node! Let’s look at compute nodes now.

10.2

First of all, we need compute nodes to mount those NFS directories that the head node exports. This can be done by adding the following lines to `/etc/fstab` (<https://linux.die.net/man/5/fstab>):

```
head-node:/gnu/store    /gnu/store    nfs    defaults,_netdev,vers=3 0 0
```

```
head-node:/var/guix /var/guix nfs defaults,_netdev,vers=3 0 0
head-node:/var/log/guix /var/log/guix nfs defaults,_netdev,vers=3 0 0
```

... where *head-node* is the name or IP address of your head node. From there on, assuming the mount points exist, you should be able to mount each of these on the compute nodes.

Next, we need to provide a default `guix` command that users can run when they first connect to the cluster (eventually they will invoke `guix pull`, which will provide them with their “own” `guix` command). Similar to what the binary installation script did on the head node, we’ll store that in `/usr/local/bin`:

```
mkdir -p /usr/local/bin
ln -s /var/guix/profiles/per-user/root/current-guix/bin/guix \
    /usr/local/bin/guix
```

We then need to tell `guix` to talk to the daemon running on our master node, by adding these lines to `/etc/profile`:

```
GUIX_DAEMON_SOCKET="guix://head-node"
export GUIX_DAEMON_SOCKET
```

To avoid warnings and make sure `guix` uses the right locale, we need to tell it to use locale data provided by Guix (see Section “Application Setup” in *GNU Guix Reference Manual*):

```
GUIX_LOCPATH=/var/guix/profiles/per-user/root/guix-profile/lib/locale
export GUIX_LOCPATH

# Here we must use a valid locale name. Try "ls $GUIX_LOCPATH/*"
# to see what names can be used.
LC_ALL=fr_FR.utf8
export LC_ALL
```

For convenience, `guix` package automatically generates `~/.guix-profile/etc/profile`, which defines all the environment variables necessary to use the packages—`PATH`, `C_INCLUDE_PATH`, `PYTHONPATH`, etc. Likewise, `guix pull` does that under `~/.config/guix/current`. Thus it’s a good idea to source both from `/etc/profile`:

```
for GUIX_PROFILE in "$HOME/.config/guix/current" "$HOME/.guix-profile"
do
    if [ -f "$GUIX_PROFILE/etc/profile" ]; then
        . "$GUIX_PROFILE/etc/profile"
    fi
done
```

Last but not least, Guix provides command-line completion notably for Bash and zsh. In `/etc/bashrc`, consider adding this line:

```
. /var/guix/profiles/per-user/root/current-guix/etc/bash_completion.d/guix
Voilà!
```

You can check that everything’s in place by logging in on a compute node and running:

```
guix install hello
```

The daemon on the head node should download pre-built binaries on your behalf and unpack them in `/gnu/store`, and `guix install` should create `~/.guix-profile` containing the `~/.guix-profile/bin/hello` command.

10.3

Guix requires network access to download source code and pre-built binaries. The good news is that only the head node needs that since compute nodes simply delegate to it.

It is customary for cluster nodes to have access at best to a *white list* of hosts. Our head node needs at least `ci.guix.gnu.org` in this white list since this is where it gets pre-built binaries from by default, for all the packages that are in Guix proper.

Incidentally, `ci.guix.gnu.org` also serves as a *content-addressed mirror* of the source code of those packages. Consequently, it is sufficient to have *only* `ci.guix.gnu.org` in that white list.

Software packages maintained in a separate repository such as one of the various HPC channels (<https://hpc.guix.info/channels>) are of course unavailable from `ci.guix.gnu.org`. For these packages, you may want to extend the white list such that source and pre-built binaries (assuming this-party servers provide binaries for these packages) can be downloaded. As a last resort, users can always download source on their workstation and add it to the cluster's `/gnu/store`, like this:

```
GUIX_DAEMON_SOCKET=ssh://compute-node.example.org \
  guix download http://starp.gforge.inria.fr/files/starp-1.2.3/starp-1.2.3.tar.gz
```

The above command downloads `starp-1.2.3.tar.gz` and sends it to the cluster's `guix-daemon` instance over SSH.

Air-gapped clusters require more work. At the moment, our suggestion would be to download all the necessary source code on a workstation running Guix. For instance, using the `--sources` option of `guix build` (see Section “Invoking guix build” in *GNU Guix Reference Manual*), the example below downloads all the source code the `openmpi` package depends on:

```
$ guix build --sources=transitive openmpi

...

/gnu/store/x17sm60fb8nxadc4qy0c7rqph499z8s-openmpi-1.10.7.tar.bz2
/gnu/store/s67jx92lpipy2nfj5cz818xv430n4b7w-gcc-5.4.0.tar.xz
/gnu/store/npw9qh8a46lrxihw9xwk0wpi3jlmjnh-gmp-6.0.0a.tar.xz
/gnu/store/hcz0f4wkdbvsdky3c0vdvcawhdkylb-mpfr-3.1.5.tar.xz
/gnu/store/y9akh452n3p4w2v631nj0inix7y0d68x-mpc-1.0.3.tar.gz
/gnu/store/6g5c35q8avfnzs3v14dzl54cmrvddjm2-glibc-2.25.tar.xz
/gnu/store/p9k48dk3dvvk7gads7fk30xc2pxsd66z-hwloc-1.11.8.tar.bz2
/gnu/store/cry9lqidwfrfmg10x389cs3syr15p13q-gcc-5.4.0.tar.xz
/gnu/store/7ak0v3rzpqm2c5q1mp3v7cj0rxz0qakf-libfabric-1.4.1.tar.bz2
/gnu/store/vh8syjrsilnbfcf582qhmvp1v3rampf-rdma-core-14.tar.gz

...
```

(In case you're wondering, that's more than 320 MiB of *compressed* source code.)

We can then make a big archive containing all of this (see Section “Invoking guix archive” in *GNU Guix Reference Manual*):

```
$ guix archive --export \
  `guix build --sources=transitive openmpi` \
```

```
> openmpi-source-code.nar
```

... and we can eventually transfer that archive to the cluster on removable storage and unpack it there:

```
$ guix archive --import < openmpi-source-code.nar
```

This process has to be repeated every time new source code needs to be brought to the cluster.

As we write this, the research institutes involved in Guix-HPC do not have air-gapped clusters though. If you have experience with such setups, we would like to hear feedback and suggestions.

10.4

A common concern of sysadmins' is whether this is all going to eat a lot of disk space. If anything, if something is going to exhaust disk space, it's going to be scientific data sets rather than compiled software—that's our experience with almost ten years of Guix usage on HPC clusters. Nevertheless, it's worth taking a look at how Guix contributes to disk usage.

First, having several versions or variants of a given package in `/gnu/store` does not necessarily cost much, because `guix-daemon` implements deduplication of identical files, and package variants are likely to have a number of common files.

As mentioned above, we recommend having a cron job to run `guix gc` periodically, which removes *unused* software from `/gnu/store`. However, there's always a possibility that users will keep lots of software in their profiles, or lots of old generations of their profiles, which is “live” and cannot be deleted from the viewpoint of `guix gc`.

The solution to this is for users to regularly remove old generations of their profile. For instance, the following command removes generations that are more than two-month old:

```
guix package --delete-generations=2m
```

Likewise, it's a good idea to invite users to regularly upgrade their profile, which can reduce the number of variants of a given piece of software stored in `/gnu/store`:

```
guix pull
guix upgrade
```

As a last resort, it is always possible for sysadmins to do some of this on behalf of their users. Nevertheless, one of the strengths of Guix is the freedom and control users get on their software environment, so we strongly recommend leaving users in control.

10.5 Security Considerations

On an HPC cluster, Guix is typically used to manage scientific software. Security-critical software such as the operating system kernel and system services such as `sshd` and the batch scheduler remain under control of sysadmins.

The Guix project has a good track record delivering security updates in a timely fashion (see Section “Security Updates” in *GNU Guix Reference Manual*). To get security updates, users have to run `guix pull && guix upgrade`.

Because Guix uniquely identifies software variants, it is easy to see if a vulnerable piece of software is in use. For instance, to check whether the glibc 2.25 variant without the mitigation patch against “Stack Clash (<https://www.qualys.com/2017/06/19/stack-clash/stack-clash.txt>)”, one can check whether user profiles refer to it at all:

```
guix gc --referrers /gnu/store/...-glibc-2.25
```

This will report whether profiles exist that refer to this specific glibc variant.

11 Guix

Since Guix does not handle packaging, system configuration and services the way other (more “classical”) distributions do, some workflows tend to unfold slightly different as we are used to and need slight adjustment. This chapter intends to help with such manners.

11.1 Postgres Cuirass

With the deprecation of the default value for the postgres package in postgresql-configuration (see b93434e656eba4260df82158a96c295000d3ff44), system upgrades need some manual action before they can take place. Here’s a handy guide on how to.

Please note that this is a straight-forward way for smaller datasets. For larger databases `pg_upgrade` (<https://www.postgresql.org/docs/current/pgupgrade.html>) may be the better choice. Handling the service and system upgrade as described in this guide still applies, though.

1. Stop and disable cuirass.

Prevent the service from starting and failing after a reconfiguration:

```
sudo herd stop cuirass && sudo herd disable cuirass
```

2. Dump the database contents.

```
sudo su - postgres -s /bin/sh -c pg_dumpall > /tmp/pg.dump
```

3. Add or alter the postgres service.

Depending on whether your postgres service is defined implicitly (through the dependency from the cuirass service) or its own entry in your operating system’s (`services`) property, you need to either add or alter the already existing configuration to reflect your intended version upgrade.

Be careful not to upgrade directly to postgres-16 – cuirass service for some reason doesn’t like that. I had to find and purge the relevant files and then re-initialize after a failed upgrade to postgres 16.

```
(service postgresql-service-type
  (postgresql-configuration
    (postgresql (@ (gnu packages databases) postgresql-15))))
```

Note: If you for some reason didn’t read the text here but somewhat blindly followed the examples and *did upgrade to 16*, here’s how you reset the state:

1. Delete the database instance files.

They default to live under `/var/lib/postgres/data`.

2. Re-initialize postgres.

```
sudo su - postgres -s /bin/sh -c 'pg_ctl init -D /var/lib/postgres/data'■
```

4. Reconfigure your system.

```
sudo guix system reconfigure path/to/your/altered/config.scm
```

5. Restore database contents.

```
sudo su - postgres -s /bin/sh -c 'psql -d postgres -f /tmp/pg.dump'
```

6. Enable and start the service.

```
sudo herd enable cuirass
sudo herd start cuirass
```

12

Guix is based on the Nix package manager (<https://nixos.org/nix/>), which was designed and implemented by Eelco Dolstra, with contributions from other people (see the `nix/AUTHORS` file in Guix.) Nix pioneered functional package management, and promoted unprecedented features, such as transactional package upgrades and rollbacks, per-user profiles, and referentially transparent build processes. Without this work, Guix would not exist.

, Nix, Nixpkgs and NixOS, Guix.

GNU Guix itself is a collective work with contributions from a number of people. See the `AUTHORS` file in Guix for more information on these fine people. The `THANKS` file lists people who have helped by reporting bugs, taking care of the infrastructure, providing artwork and themes, making suggestions, and more—thank you!

This document includes adapted sections from articles that have previously been published on the Guix blog at <https://guix.gnu.org/blog> and on the Guix-HPC blog at <https://hpc.guix.info/blog>.

Appendix A GNU

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

<https://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released

under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

The “publisher” means any person or entity that distributes copies of the Document to the public.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any,

be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their

titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements."

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <https://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

“Massive Multiauthor Collaboration Site” (or “MMC Site”) means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A “Massive Multiauthor Collaboration” (or “MMC”) contained in the site means any set of copyrightable works thus published on the MMC site.

“CC-BY-SA” means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

“Incorporate” means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is “eligible for relicensing” if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.3
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts. A copy of the license is included in the section entitled ``GNU
Free Documentation License''.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with...Texts.” line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

..... 9

..... 80

..... 98

..... 92

, GNU Free Documentation License 100

2

2FA, two-factor authentication 35

A

avoid ABI mismatch, container 56

B

bluetooth, ALSA configuration 50

C

container networking 60

D

development, with Guix 74

disabling yubikey OTP 36

disk usage, on a cluster 96

dynamic DNS, DDNS 37

E

exiting a container 55

exposing directories, container 55

G

G-, 2

gexps, syntax 2

H

hide system libraries, container 56

high-performance computing, HPC 92

HPC, high-performance computing 92

K

kimsufi, Kimsufi, OVH 44

L

libvirt, virtual network switch 63

linode, Linode 40

lock files, for reproducibility 89

M

mapping locations, container 55

mpd 50

music server, headless 50

N

Network bridge interface 62

networking, bridge 62

networking, virtual bridge 63

nginx, lua, openresty, resty 49

P

packaging 5

Q

qemu, network bridge 62

R

reproducible research 88

S

S-..... 2

Scheme, 1

security key, configuration 35

security, on a cluster 96

sessionlock 40

sharing directories, container 55

software development, with Guix 74

stumpwm 39

stumpwm fonts 39

sysadmin 98

U

U2F, Universal 2nd Factor 35

V

Virtual network bridge interface 63

W

wm 39

Y

yubikey, keepassxc integration 36